Recommender Systems

Lecture 1

Today's material

Lecture 1

General introduction

RecSys Evaluation

Lecture 2

Collaborative Filtering via Matrix Factorization

About me

- Personalization Tech Group Lead at AIRI
- Associate Professor at HSE, Senior Researcher at MTML Lab
- Senior Research Scientist at Computational Intelligence Lab, Skoltech;

PhD in Data Science:
https://www.skoltech.ru/en/2018/09/phd-thesis-defense-evgeny-frolov/

Previously:

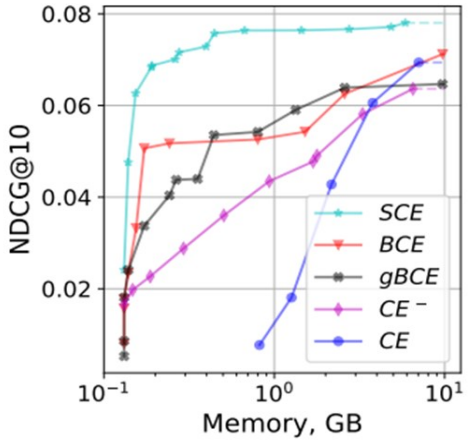
- Sberbank AI Laboratory, head of recsys department
- Graduated from MSU, Chair of Polymer and Crystal Physics
- 5 years of professional experience in IT

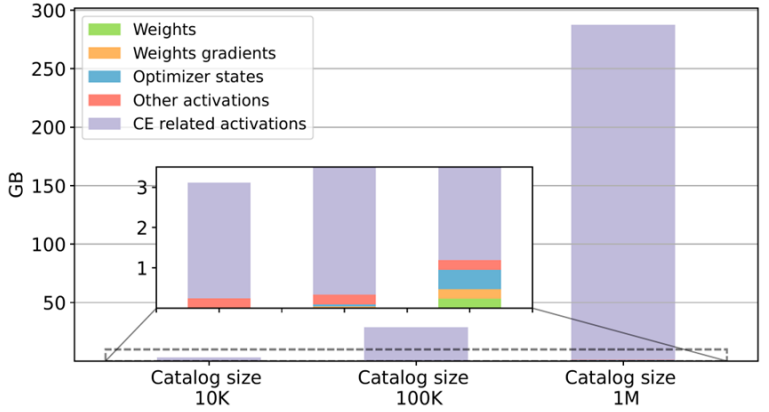
Personalization Technologies Lab

Accurate Recommendations with Hyperbolic Geometry

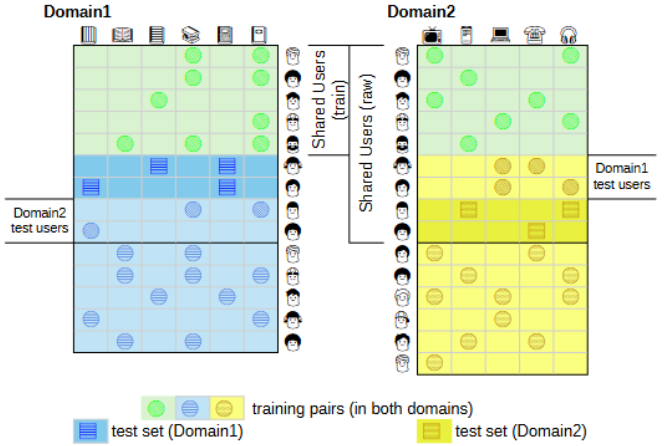
(a) M. C. Eschers Circle Limit III, 1959

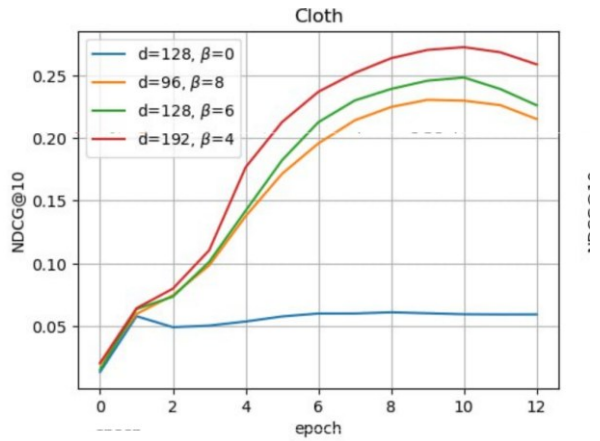
Scalable Cross-Entropy Loss for Sequential Recommendations with Large Item Catalogs





Transfer Learning via Cross-Domain Recommendations





Reading

Main book:

- **Personalized Machine Learning**, Julian McAuley, 2022 (in press); https://cseweb.ucsd.edu/~jmcauley/pml/pml_book.pdf

Additional reading (in no particular order):

- **Recommender Systems: A Primer,** 2023; P. Castells, D. Jannach
- **Recommender Systems. The Textbook**, 2016; Charu C. Aggarwal
- **Recommender Systems Handbook,** 2022, 3rd edition; F. Ricci, L. Rokach, B. Shapira

What is a recommender system?

Examples:

- Amazon
- Netflix
- Pandora
- Spotify
- Social platforms
- etc.

Many different areas: e-commerce, news, tourism, entertainment, education…

Goal: predict user preferences given some prior information on user behavior.

Drug Discovery and Recommender Systems

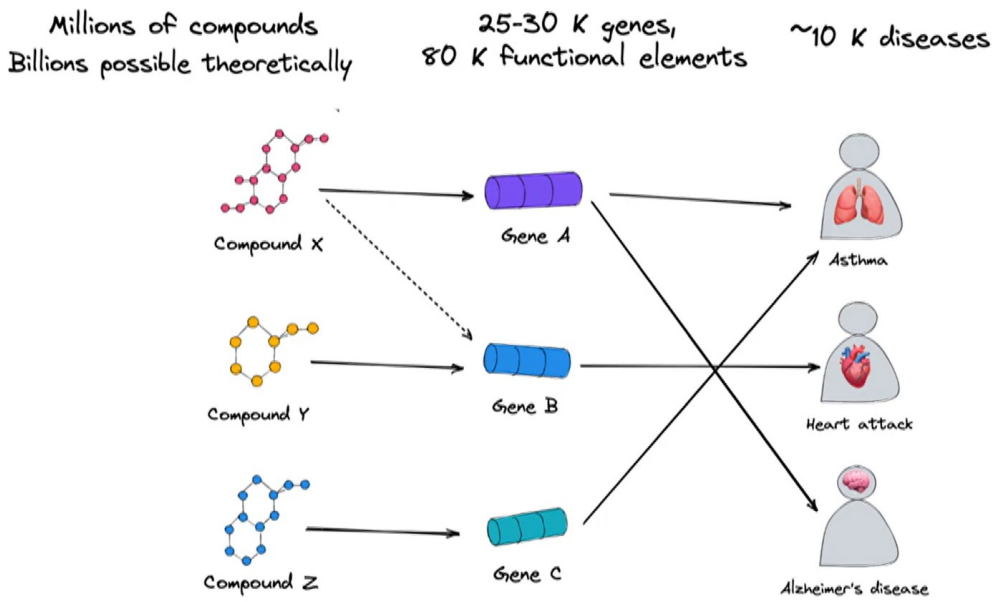


Image Source:
Gogleva, Anna, Eliseo Papa, Erik Jansson, and Greet De Baets. "Drug Discovery as a Recommendation Problem: Challenges and Complexities in Biological Decisions." In *Fifteenth ACM Conference on Recommender Systems*, pp. 548-550. 2021.

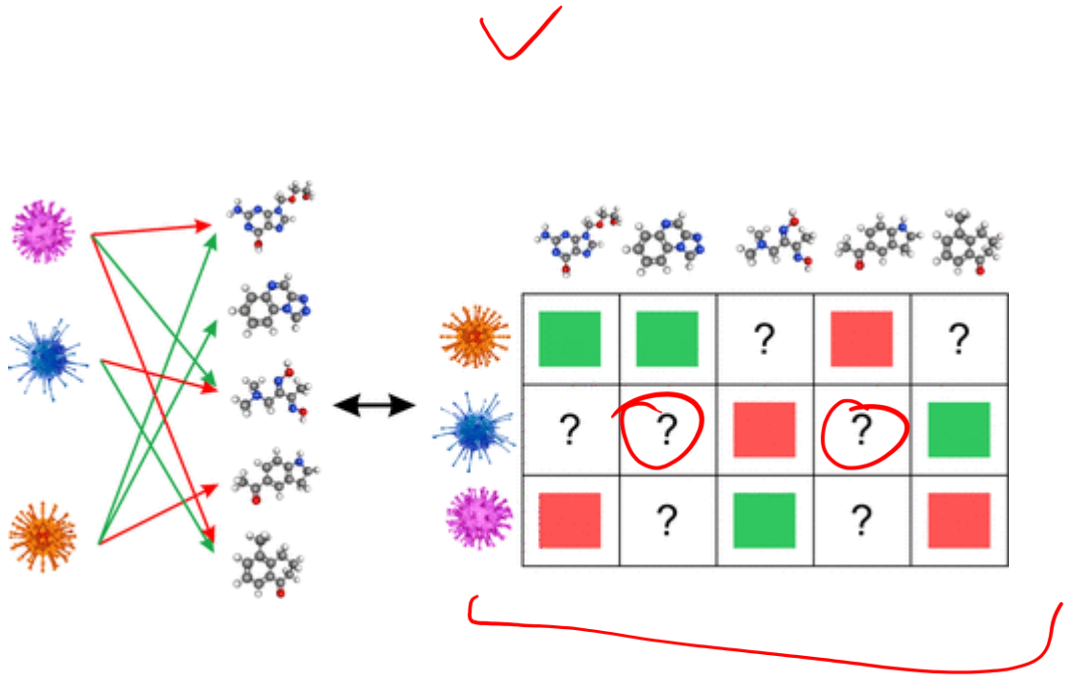


Image Source:
Sosnina, Ekaterina A., Sergey Sosnin, Anastasia A. Nikitina, Ivan Nazarov, Dmitry I. Osolodkin, and Maxim V. Fedorov. "Recommender systems in antiviral drug discovery." *ACS omega* 5, no. 25 (2020): 15039-15051.

Evaluation of Recommender Systems

Types of evaluation

Offline Evaluation

- straightforward
- doesn't require real users
- can be used to benchmark the generalizability of models
- may not correlate with online/business metrics

Online Evaluation

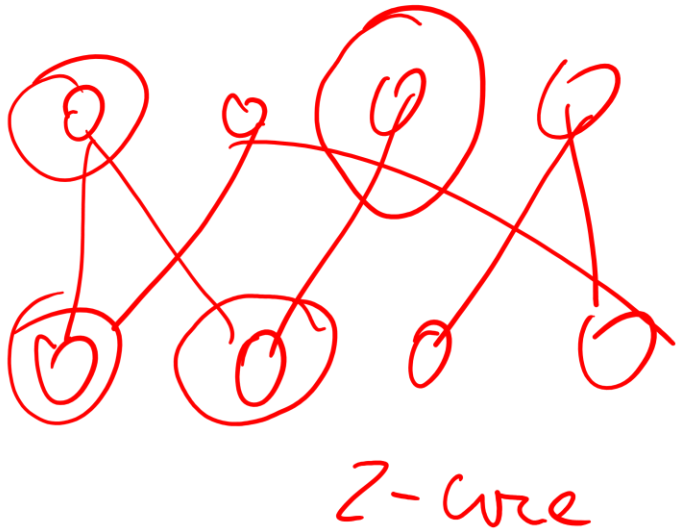
- real users of a commercial recsys
- resembles clinical trials
- no public access, limited to specific domain and data

User studies

- users are recruited for an evaluation task
- can be performed by both scientists and practitioners
- can be biased

Factors influencing quality of recommendations

- Length of user profile
  - active users vs bots
  - “warm” vs “hot” users
- Concentration of popular items in user history
  - short head vs long tail items
- p-core filtering



p-core filtering effects

Table 1: Statistics of datasets.

Dataset		ML-1M	Lastfm	Yelp	Epinions	Book-X	AMZe
origin	#User	6,038	1,892	1,326,101	22,164	105,283	4,201,696
	#Item	3,533	17,632	174,567	296,277	340,556	476,002
	#Record	575,281	92,834	5,261,669	922,267	1,149,780	7,824,482
	Density	2.697e-2	2.783e-3	2.273e-5	1.404e-4	3.207e-5	3.912e-6
5-filter	#User	6,034	1,874	227,109	21,995	22,072	253,994
	#Item	3,125	2,828	123,985	31,678	43,748	145,199
	#Record	574,376	71,411	3,419,587	550,117	623,405	2,109,869
	Density	3.046e-2	1.348e-2	1.214e-4	7.895e-4	6.456e-4	5.721e-5
10-filter	#User	5,950	1,867	96,168	21,111	12,720	63,161
	#Item	2,811	1,530	80,351	14,030	18,318	85,930
	#Record	571,549	62,984	2,458,153	434,162	443,196	949,416
	Density	3.412e-2	2.205e-2	3.181e-4	1.466e-3	1.902e-3	1.749e-4
Timestamp		√	×	√	√	×	√

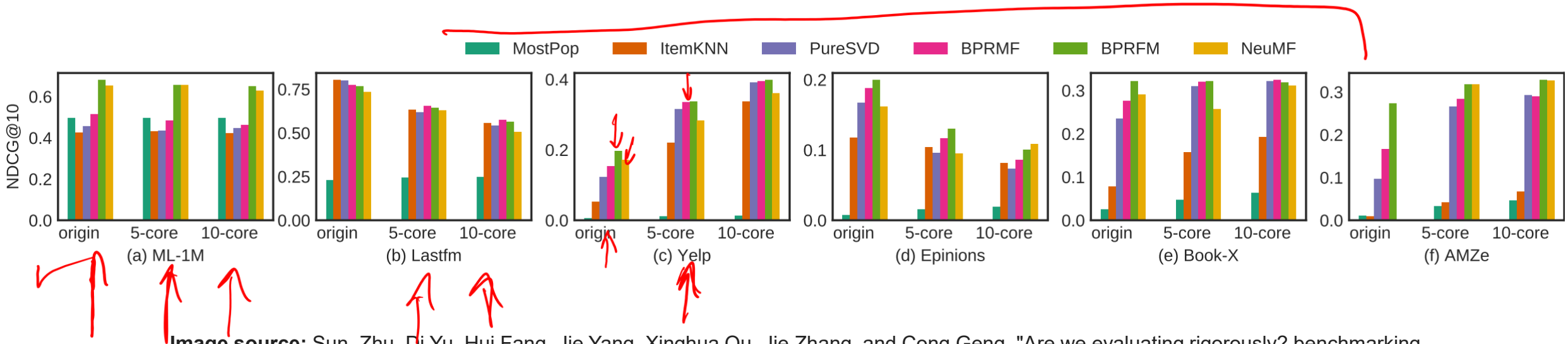


Image source: Sun, Zhu, Di Yu, Hui Fang, Jie Yang, Xinghua Qu, Jie Zhang, and Cong Geng. "Are we evaluating rigorously? benchmarking recommendation for reproducible evaluation and fair comparison." In *Fourteenth ACM conference on recommender systems*, pp. 23-32. 2020.

Offline evaluation methodologies

Can’t we just use standard ML procedures?

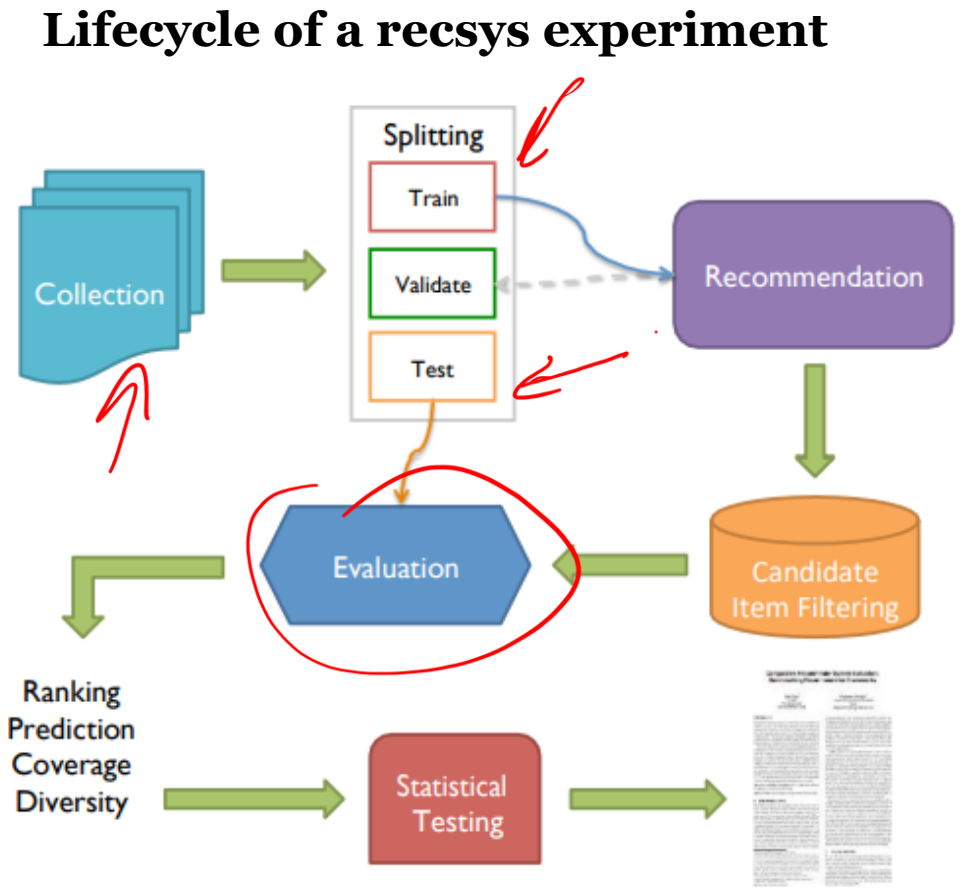
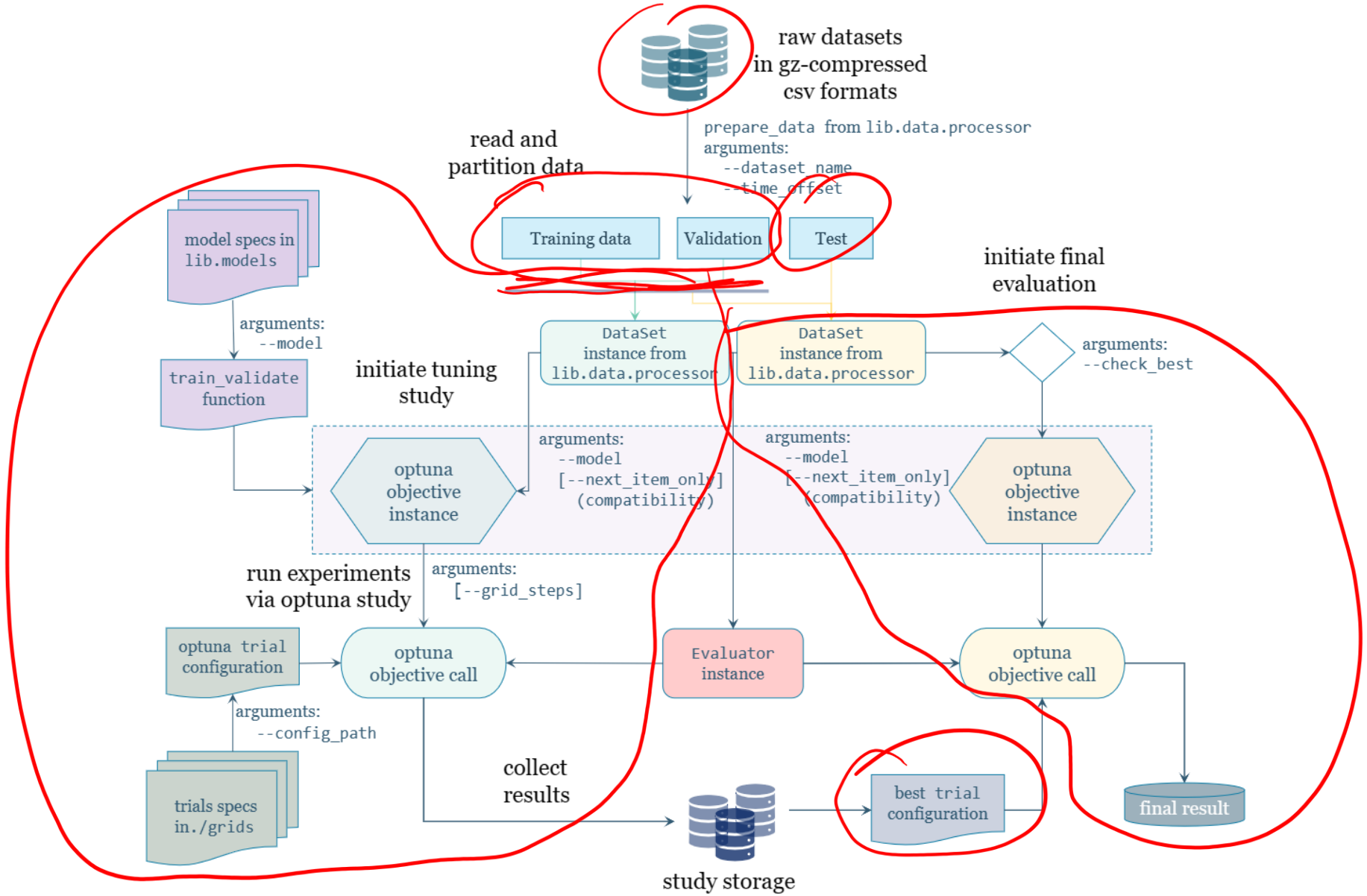
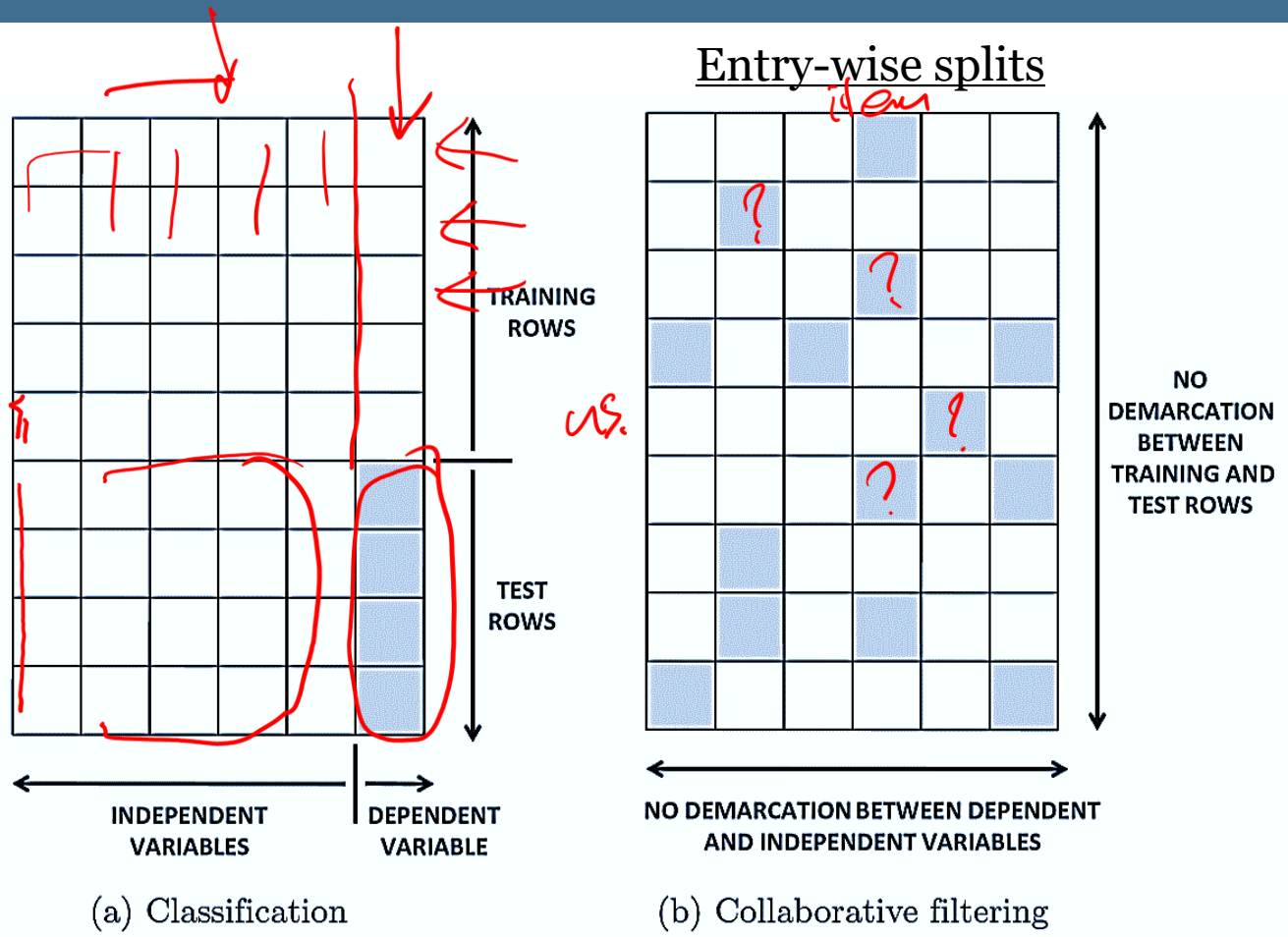


Image source: Bellogín, Alejandro, and Alan Said. "*Improving accountability in recommender systems research through reproducibility*." User Modeling and User-Adapted Interaction (2021): 1-37.

Experimentation pipeline example



Classification/Regression or Recommendation?

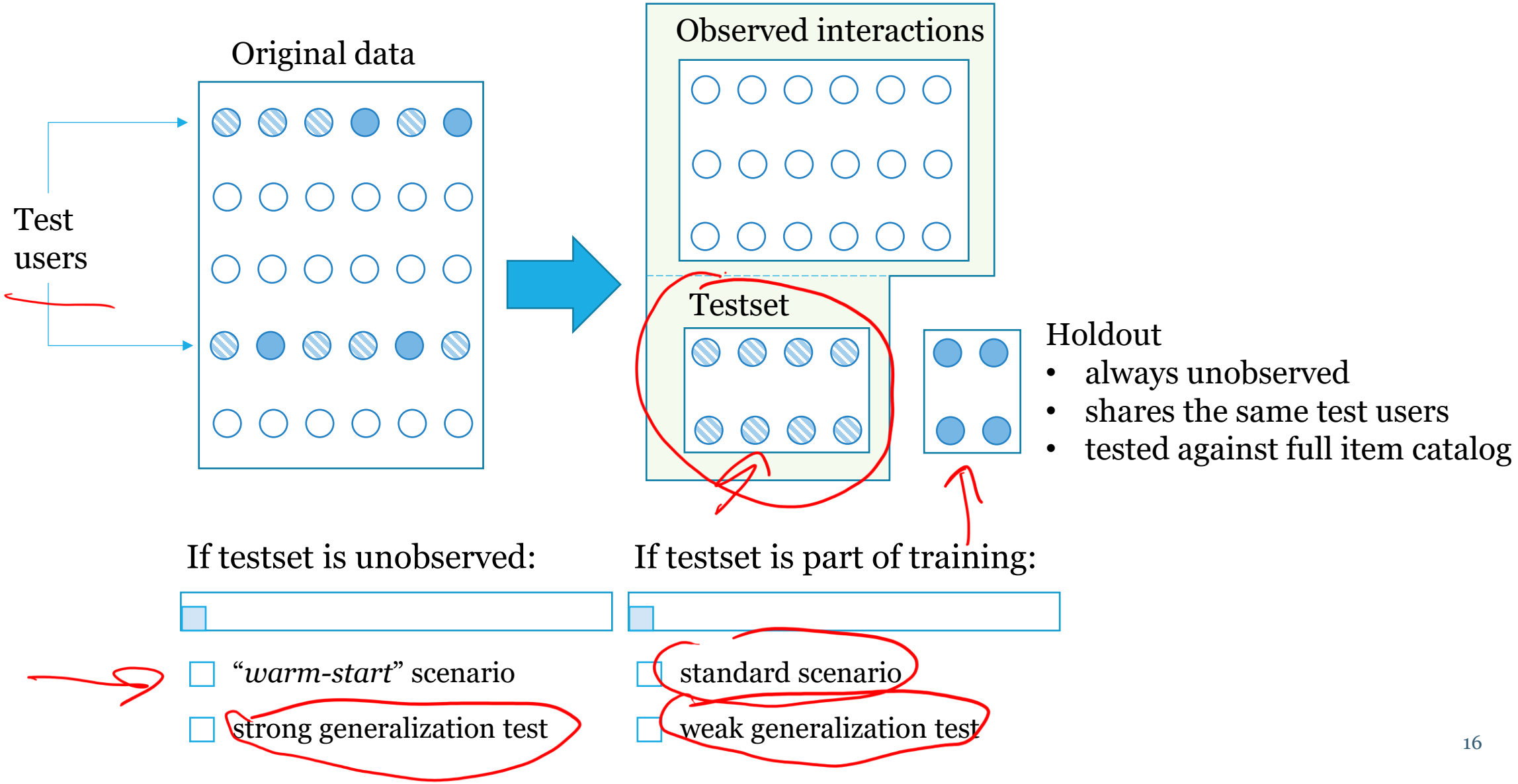


(a) Classification (b) Collaborative filtering

- generating recommendations is often viewed as a separate task
- can still be formulated as a regression/classification problem
- dealing with new entities (users, items) is less straightforward, though

Image credit: "Recommender Systems. The Textbook", Charu C. Aggarwal, 2016

User-wise data split schemes



Holdout items sampling

Strategies

- Random
- Rating-based (e.g., top-rated)
- Temporal (e.g., most recent)

next-item rec

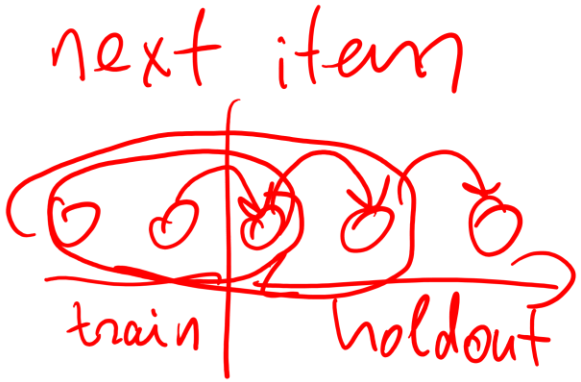
Sample size

- Fixed number of items (e.g., 1)
- Fixed percentage of items

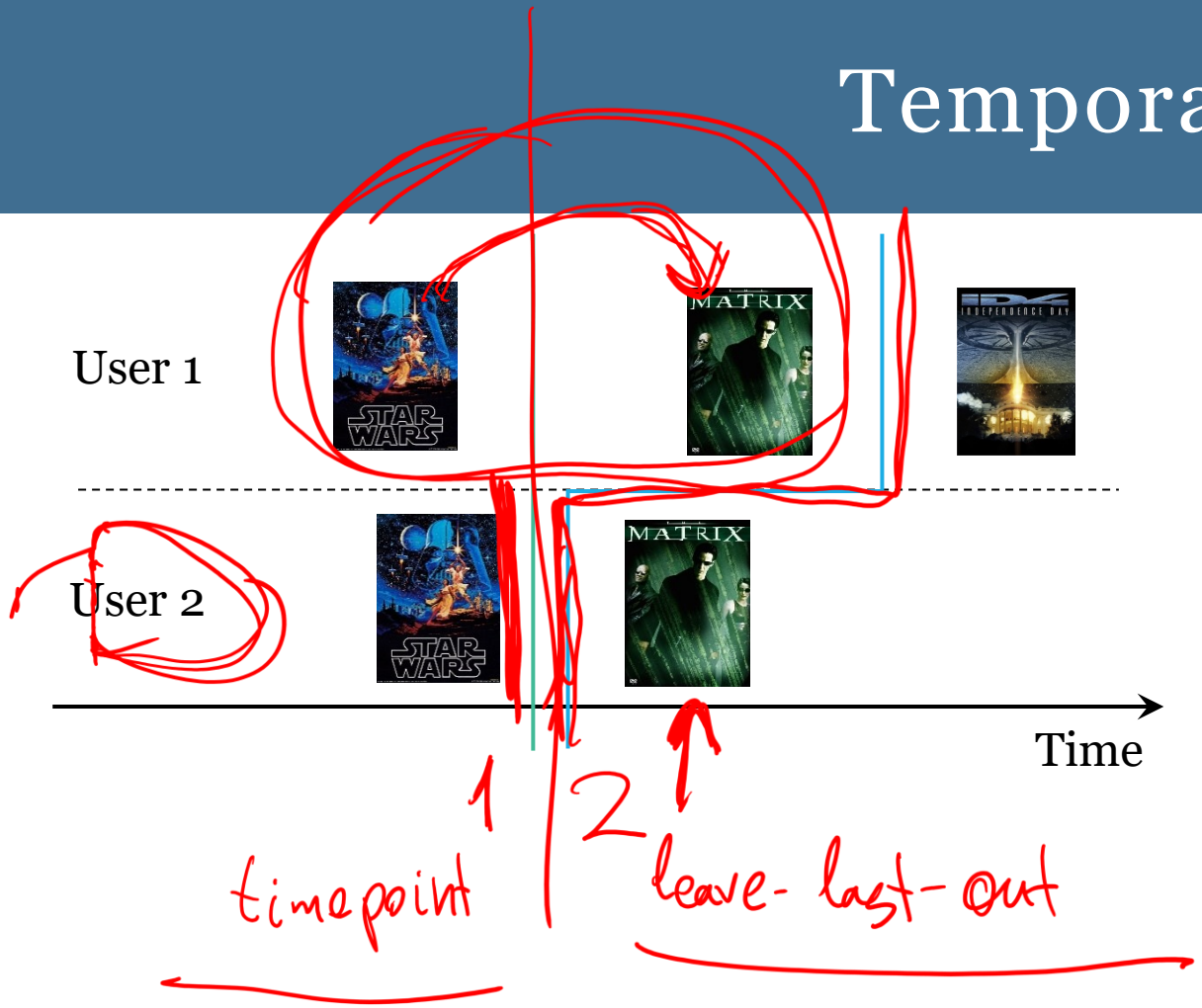
Typical data splitting strategies

- random holdout split
  - leave-%-out or leave-m-out, $m = 1, 2, \ldots$
  - use k-fold cross-validation

- temporal split
  - leave-last-out / timepoint split
  - successive evaluation



Temporal splits

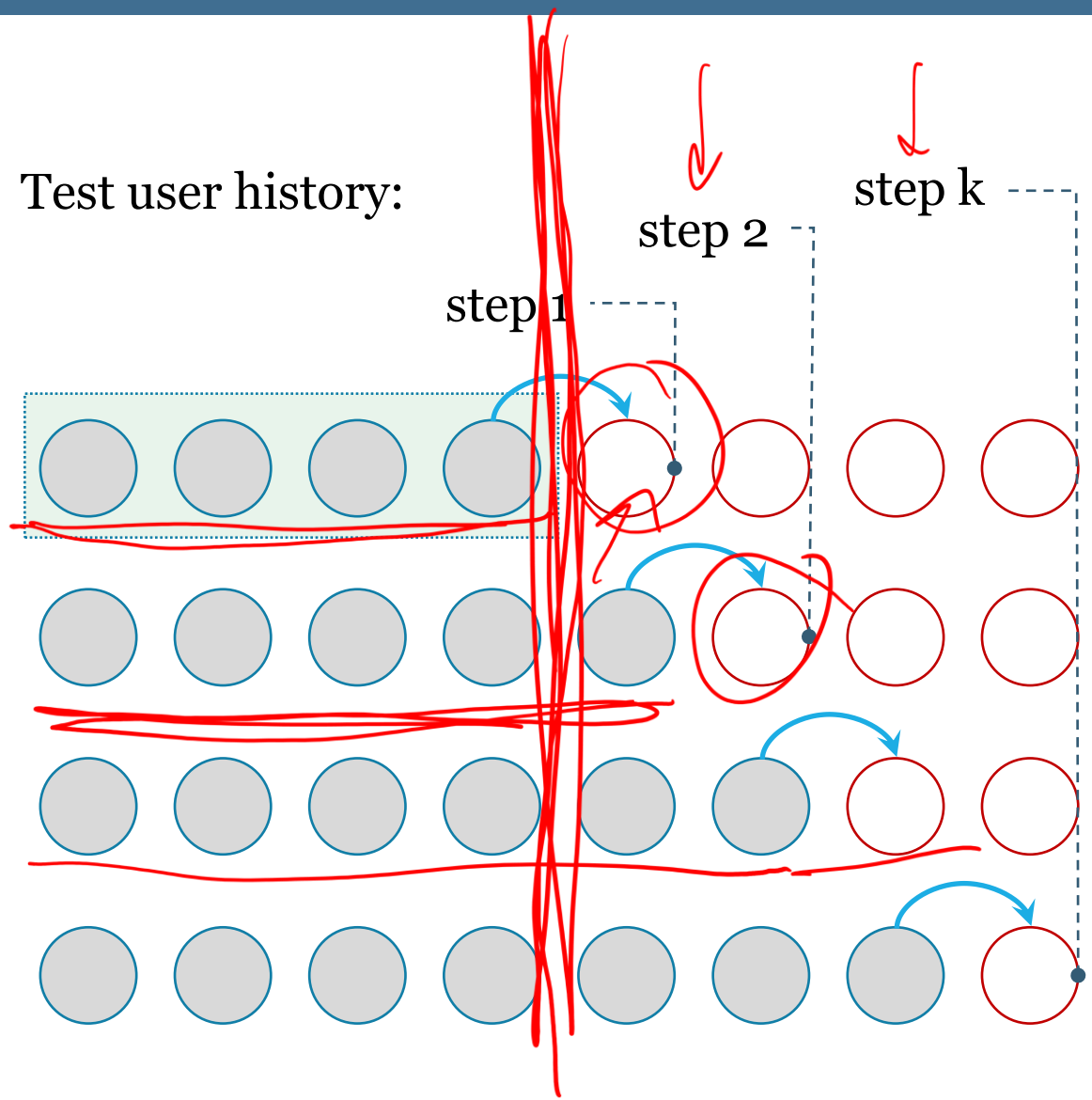


SASRec
Bert4Rec

Data split strategy	Definition of training and test instances	Local timeline	Global timeline	Data leakage
Random-split-by-ratio	Randomly sample a percentage of user-item interactions as test instances; the remaining are training instances.	No	No	Yes
Random-split-by-user	Randomly sample a percentage of users, and take all their interactions as test instances; the remaining instances from other users are training instances.	No	No	Yes
Leave-one-out-split	Take each user's last interaction as a test instance; all remaining interactions are training instances.	Yes	No	Yes
Split-by-timepoint	All interactions after a time point are test instances; interactions before this time point are training instances.	No	Yes	No

Table source: Ji, Yitong, Aixin Sun, Jie Zhang, and Chenliang Li. "A critical study on data leakage in recommender system offline evaluation." *arXiv preprint arXiv:2010.11060* (2020).

Successive next-item revealing scheme



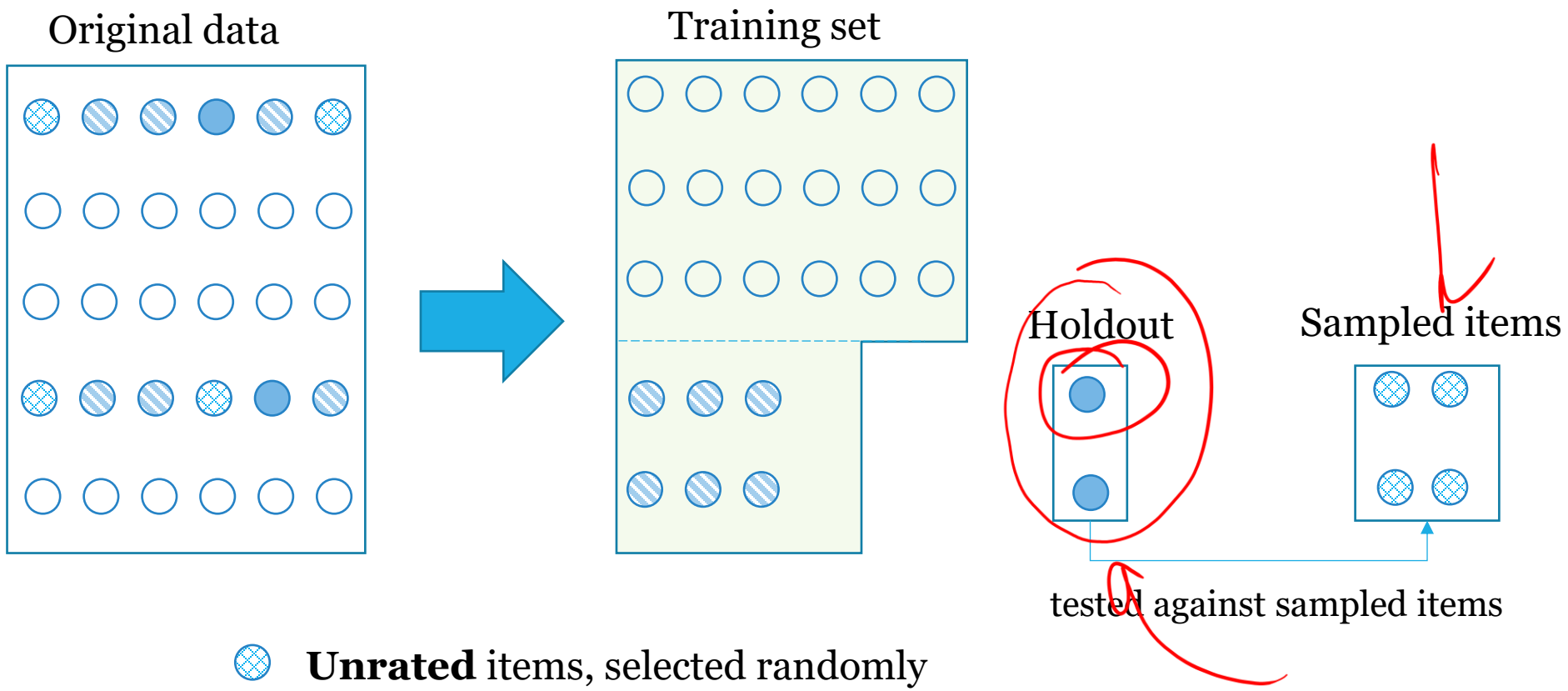
Initial training data

Target items

Revealed items

Prediction

Target items sampling



Unrated items, selected randomly

Target items sampling issues

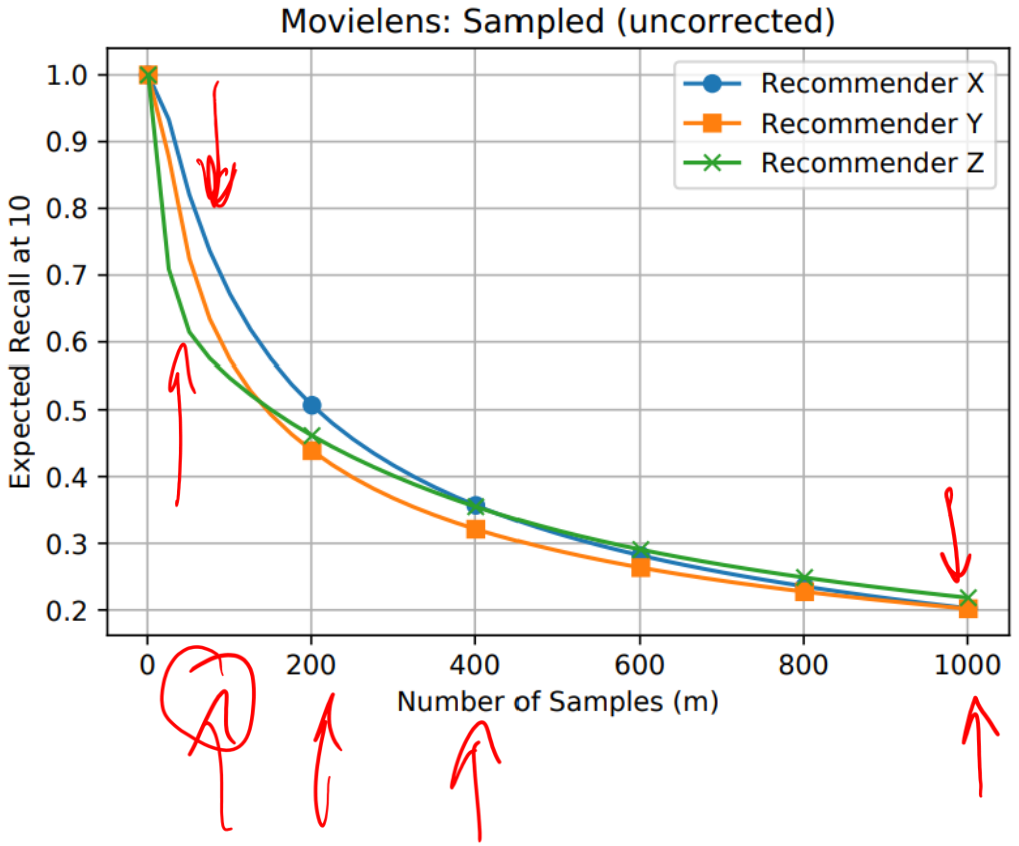


Image source: Krichene, Walid, and Steffen Rendle. "On sampled metrics for item recommendation." In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pp. 1748-1757. 2020.

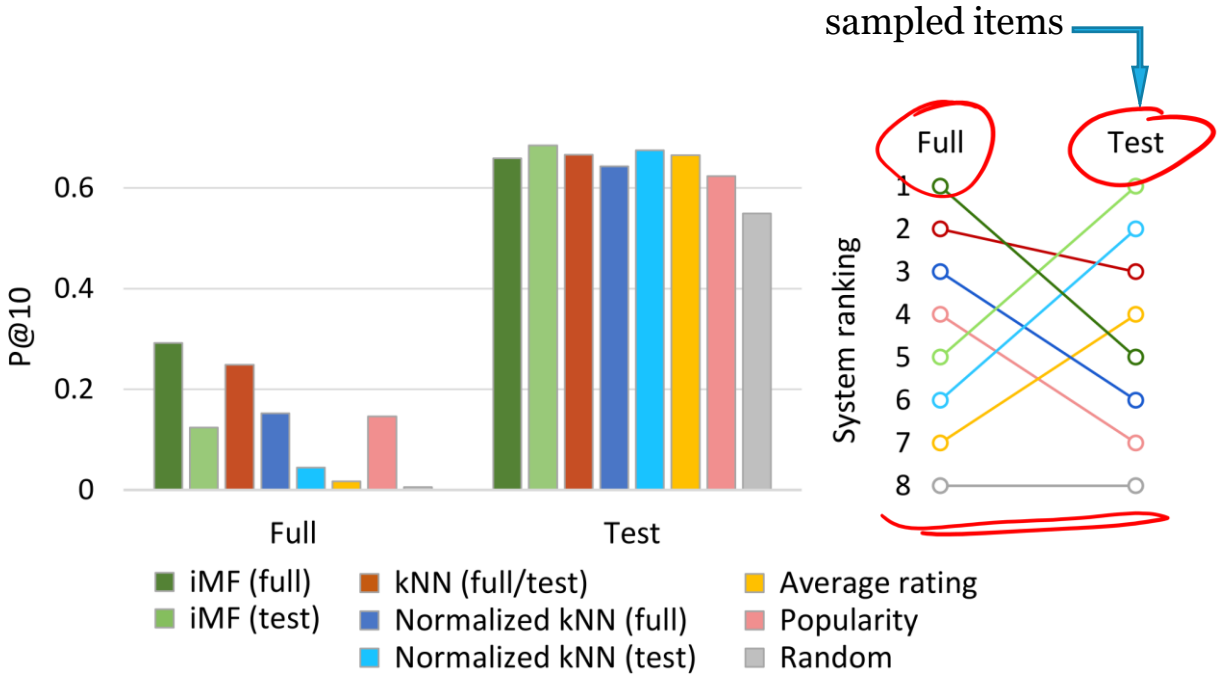


Image source: Cañamares, Rocío, and Pablo Castells. "On target item sampling in offline recommender system evaluation." In *Fourteenth ACM Conference on Recommender Systems*, pp. 259-268. 2020.

Methodological chaos

Table 1. Overview of data splitting strategies reported in the literature, as well as the dataset(s) those papers use.

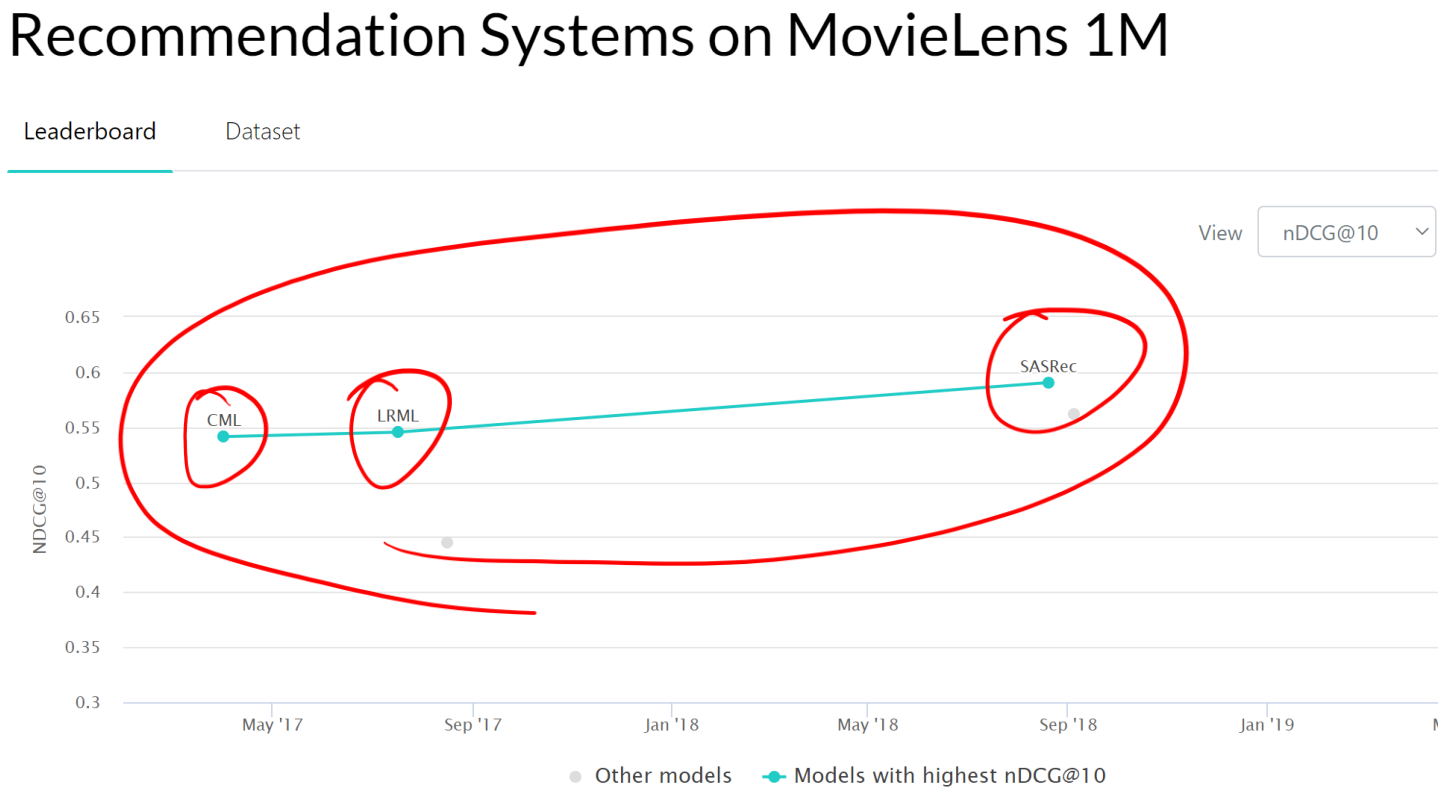
Model	Leave One Last		Temporal Split		Random Split	User Split	Used Datasets
	Item	Basket/Session	User-based	Global			
BPR [13] (2009)	×	×	×	×	√	×	N
FPMC [14] (2010)	×	√	×	×	×	×	-
NeuMF [4] (2017)	√	×	×	×	×	×	M1, P
VAECF [9] ((2018))	×	×	√	×	×	√	M2, N
Triple2vec [22] (2018)	×	√	×	×	×	×	I, D
SARRec [6] (2018)	√	×	×	×	×	×	A, M1
CTRec [1] (2019)	√	√	√	×	×	×	T, A
SVAE [17] (2019)	×	×	√	×	×	√	M1, N
BERT4Rec [20](2019)	√	×	×	×	×	×	A, M1, M2
NGCF [24] (2019)	×	×	×	×	√	×	A, G, Y
VBCAR [11] (2019)	×	×	×	√	×	×	I
KGAT [23] (2019)	×	×	√	×	×	×	A, Y
Set2Set [5] (2019)	×	×	√	×	×	×	T, D
DCRL [25] (2019)	×	×	×	√	×	×	M2, G
TiSASRec [8] (2020)	√	×	×	×	×	×	M1, A
JSR [26] (2020)	√	×	×	×	×	×	M2, A
HashGNN [21] (2020)	×	×	×	×	√	×	M2, A

M1: Movielens-1M, M2: Movielens-20M, **T**: Tafeng, D: Dunnhumby
G: Gowalla, I: Instacart N: Netflix, A: Amazon, Y: Yelp, P: Pinterest

Table from: Meng, Zaiqiao, Richard McCreadie, Craig Macdonald, and Iadh Ounis. "Exploring data splitting strategies for the evaluation of recommendation models." In *Fourteenth ACM conference on recommender systems*, pp. 681-686. 2020.

Public leaderboards are (mostly) inconsistent

Example: Papers With Code
https://paperswithcode.com/sota/collaborative-filtering-on-movielens-1m



CML paper:

4.2 Evaluation Methodology

We divide each user's ratings into training/validation/test sets in a 60%/20%/20% split. Users who have less than 5 ratings are only included in the training set. The predicted ranking is evaluated on recall rates for Top-K recommendations, which are widely-used performance metrics for implicit feedback [46, 47].

SASRec paper:

We followed the same preprocessing procedure from [1], [19], [21]. For all datasets, we treat the presence of a review or rating as implicit feedback (i.e., the user interacted with the item) and use timestamps to determine the sequence order of actions. We discard users and items with fewer than 5 related actions. For partitioning, we split the historical sequence $\mathcal{S}^u$ for each user u into three parts: (1) the most recent action $\mathcal{S}^u_{|\mathcal{S}^u|}$ for testing, (2) the second most recent action $\mathcal{S}^u_{|\mathcal{S}^u|-1}$ for validation, and (3) all remaining actions for training. Note that during testing, the input sequences contain training actions and the validation action.

To avoid heavy computation on all user-item pairs, we followed the strategy in [14], [48]. For each user u, we randomly sample 100 negative items, and rank these items with the ground-truth item. Based on the rankings of these 101 items, Hit@10 and NDCG@10 can be evaluated.

“The Field Needs Better, More Unified and Harder Evaluation”

Reading:

- Dacrema, Maurizio Ferrari, Simone Boglio, Paolo Cremonesi, and Dietmar Jannach. "**A troubling analysis of reproducibility and progress in recommender systems research.**" *ACM Transactions on Information Systems (TOIS)* 39, no. 2 (2021): 1-49.

- Zhang, Shuai, Lina Yao, Aixin Sun, and Yi Tay. "**Deep learning based recommender system: A survey and new perspectives.**" *ACM Computing Surveys (CSUR)* 52, no. 1 (2019): 1-38.

Metrics

Top-n recommendations task

Expected output: a ranked list of n most relevant items.

$$\text{toprec}(i, n) := \arg\max_{j}^{n} r_{ij}$$

r_{ij} - is the predicted relevance score between user i and item j

Example:

r_{ij}	0.73	0.41	0.95
item			

top-2 recommendations

leftsorted list

Metrics

Error-based

RMSE, MAE

Relevance based

precision, recall

F1-score

HR (hit rate)

Ranking-based

nDGC (normalized discounted cumulative gain)

MAP (mean average precision)

MRR (mean reciprocal rank)

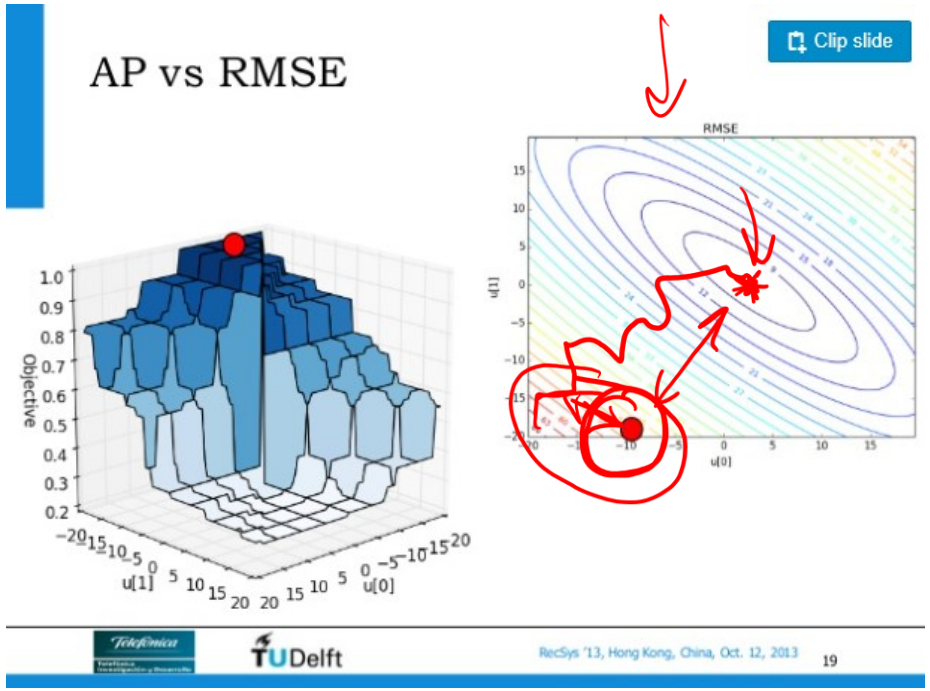
ATOP (area under the TOPK-curve)

AUC (area under the ROC-curve)

Other aspects:

- Coverage
- Novelty
- Serendipity
- Diversity
- Trust
- Utility

Error minimization vs recommendations accuracy



- Error metrics penalize errors.

vs

- Accuracy metrics reward correct answers.

General asymmetry of recommendations evaluation:
- it's useless to have low error on irrelevant items, if error is high on relevant items.

Figure credit: http://www.slideshare.net/kerveros99/learning-to-rank-for-recommender-system-tutorial-acm-recsys-2013

Metrics calculation chaos

Library	HitRate@20	MAP@20	MRR@20	NDCG@20	Precision@20	Recall@20	RocAuc
RePlay	0.475	0.039	0.186	0.093	0.058	0.096	0.283
DL RS Evaluation	1.15	0.039	0.186	0.08	0.058	0.096	0.283
DaisyRec	0.475	0.023*	0.275	0.093	0.058	0.096	fail
Beta RecSys	—	0.032	—	0.093	0.058	0.096	0.687
RecBole	0.475	0.039	0.186	0.093	0.058	0.096	0.687
Elliot	0.475	0.073	0.186	0.09	0.058	0.096	0.688
OpenRec	—	—	—	0.463	0.058	0.096	0.705
MS Recommenders	—	0.032	—	0.093	0.058	0.096	0.687
NeuRec	0.475*	0.003	0.186	0.093	0.058	0.096	—
rs_metrics	0.475	0.032	0.186	0.093	0.058	0.096	—

Table 1: Metrics calculated using different libraries.

Table source: Tamm, Y. M., Damdinov, R., & Vasilev, A. (2021). Quality metrics in recommender systems: Do We calculate metrics consistently? *RecSys 2021 - 15th ACM Conference on Recommender Systems*, 708–713.

Metrics calculation example

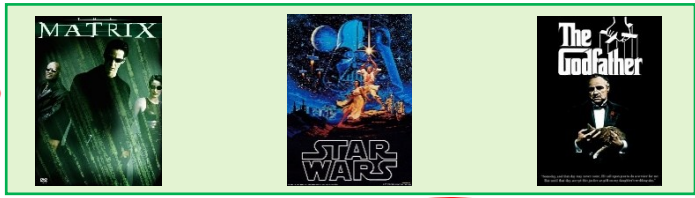
User

known user preferences

holdout

recommendations

Algorithm:



$$\text{Precision} = \frac{1}{\#(\text{test users})} \sum_{\substack{\text{test} \\ \text{users}}} \frac{\#\,(\text{recommended items} \cap \text{holdout items})}{\#\,(\text{recommended items})}$$

$$\text{Recall} = \frac{1}{\#(\text{test users})} \sum_{\substack{\text{test} \\ \text{users}}} \frac{\#\,(\text{recommended items} \cap \text{holdout items})}{\min(\#\,(\text{holdout items}), \#\,\text{recom. items})}$$

Denoted as *metric@n*, where n = #recommended items, e.g. Recall@n

Metrics calculation example

Assume #{holdout items} = const = 1, then Recall ≡ HitRate:

HitRate:

$$\mathrm{HR@}n = \frac{1}{\#(\text{test users})} \sum_{\substack{\text{test} \\ \text{users}}} \text{hit}$$

$$\text{hit} = \begin{cases} 1 & \text{if any holdout item is among } n \text{ recommended items,} \\ 0 & \text{otherwise.} \end{cases}$$

Mean Reciprocal Rank:

$$\mathrm{MRR@}n = \frac{1}{\#(\text{test users})} \sum_{\substack{\text{test} \\ \text{users}}} \text{hit} \cdot \frac{1}{\text{hit rank}}$$

hit rank = earliest position of a holdout item in the left-ordered list of n recommendations

Task

Description:

- There're 4 test users.
- Each user is presented with a list of top-4 personal recommendations.
- One user got a single correct recommendation and it's located at the 3rd position in the list.
- Other users got no correct recommendations.

Questions:

- What are HR and MRR in that case?
- How will HR change for top-5 recommendations (assuming no new correct guesses)?
- How will HR change for top-2 recommendations?

Ranking metrics: nDCG

$$DCG@n = \frac{1}{|U_{\text{test}}|} \sum_{u \in U_{\text{test}}} \sum_{i \in R_u} \frac{2^{r_{ui}} - 1}{\log_2(\text{rank}(u,i) + 1)}$$

U_{test} – set of test users
R_u – ordered set of recommended items for user u
$\text{rank}(u,i)$ – position of item i in R_u
r_{ui} - true rating of a recommended item, assuming $r_{ui} = 0$ if item is not rated

$$nDCG@n = \frac{DCG@n}{iDCG@n}$$

$iDCG = DCG$ with "ideal" (based on true ratings) ranking of items

Alternative (linear) weighting: $\frac{r_{ui}}{\log_2(\text{rank}(u,i)+1)}$

Ranking metrics: MAP

Mean Average Precision:

$$\text{MAP@}n = \frac{1}{|U_{\text{test}}|} \sum_{u \in U_{\text{test}}} \frac{1}{|R_u \cap H_u|} \sum_{i \in R_u \cap H_u} \text{P}_u\text{@rank}(u, i)$$

H_u - holdout items for user u

P_u - precision for user u with partial R_u

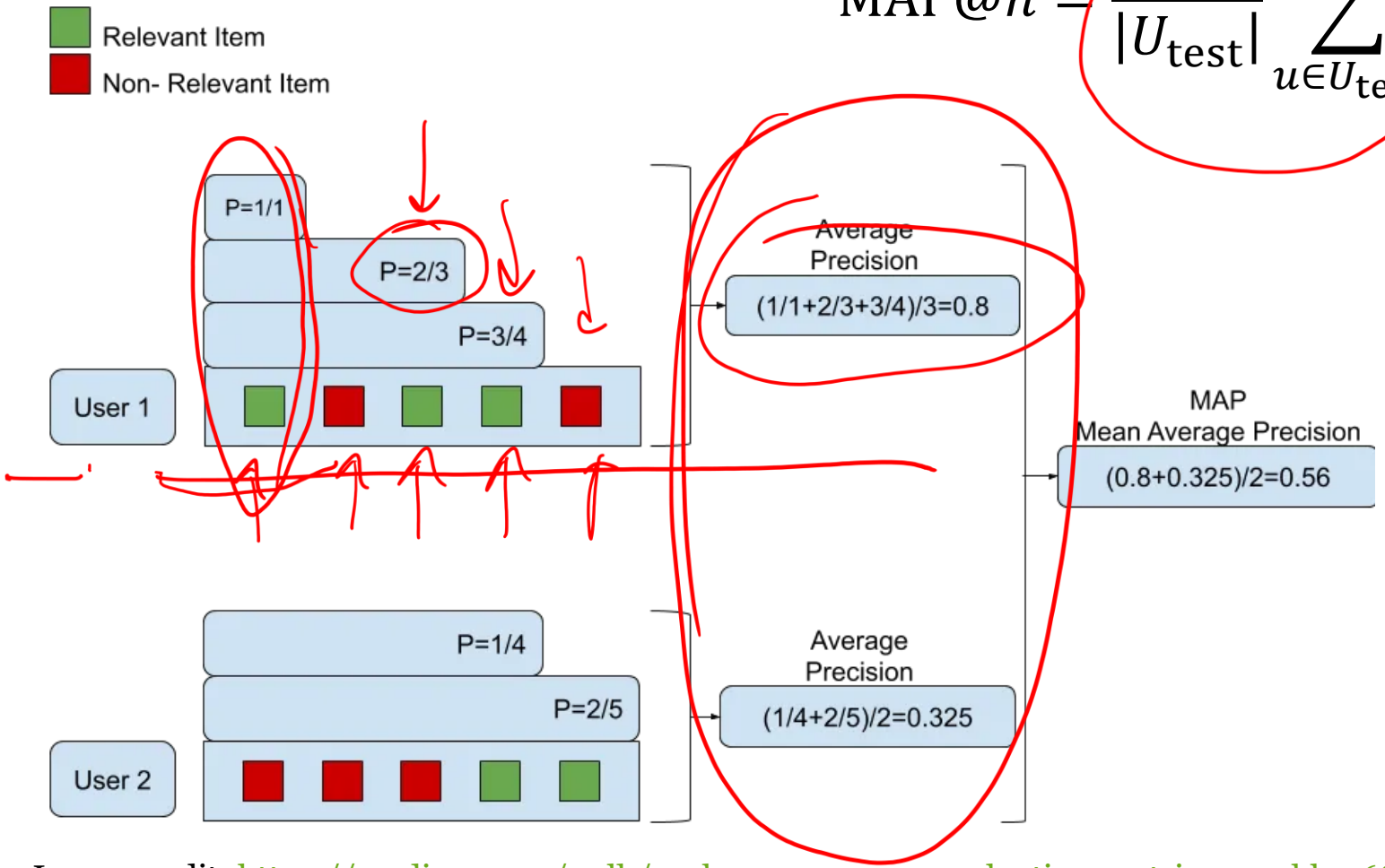


Image credit: https://medium.com/swlh/rank-aware-recsys-evaluation-metrics-5191bba16832

Useful metric: Coverage

- fraction of unique recommendations in entire item catalog:

$$\mathrm{COV@}n = \frac{\left|\bigcup_{u \in U_{\text{test}}} R_u\right|}{|I|}$$

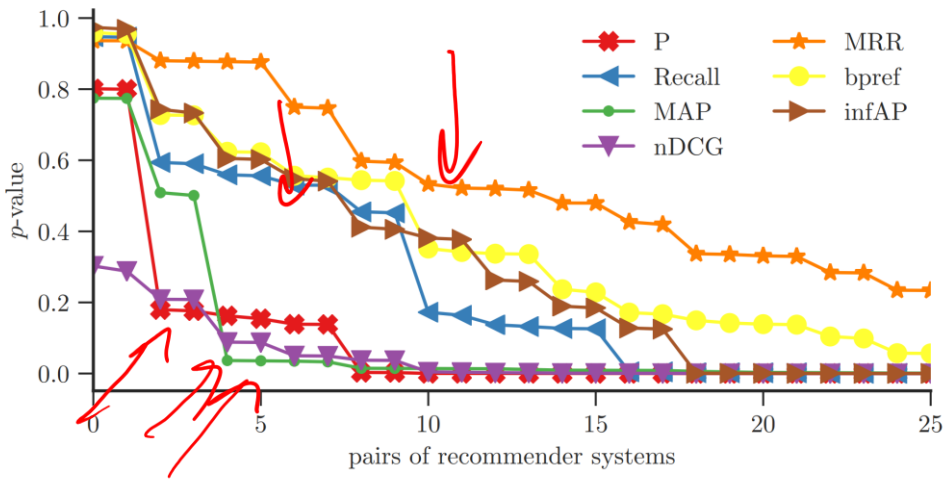
I – set of all items

- simple metric to measure diversity of recommendations
- enables quick analysis of recommendations adequacy
  - lower values – potentially too obvious recommendations
  - high values – potentially too random recommendations

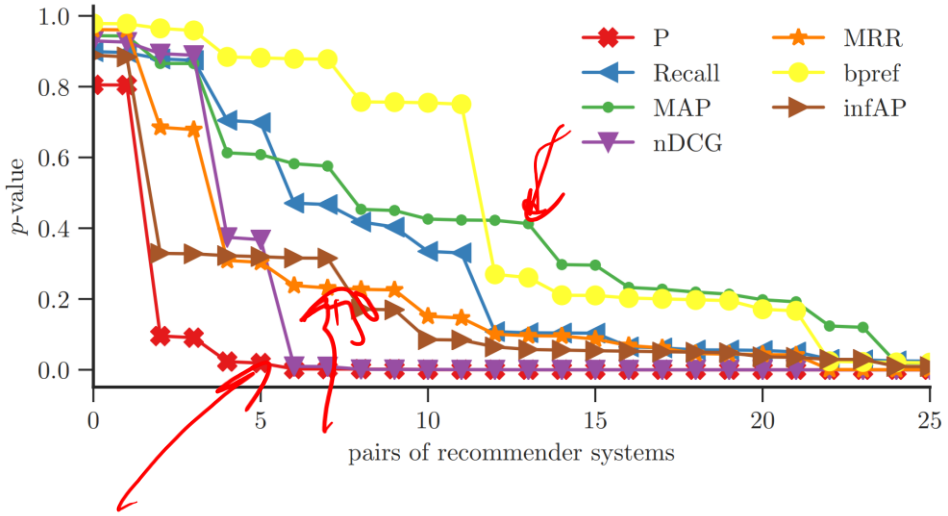
A note on discriminative power of metrics

Protocol:

- compare two recommendation techniques
- variation in the metric values is expected to indicate a statistically significant difference
- use statistical tests to measure the significance (via *p*-value, the lower the better)



a: MovieLens 1M.



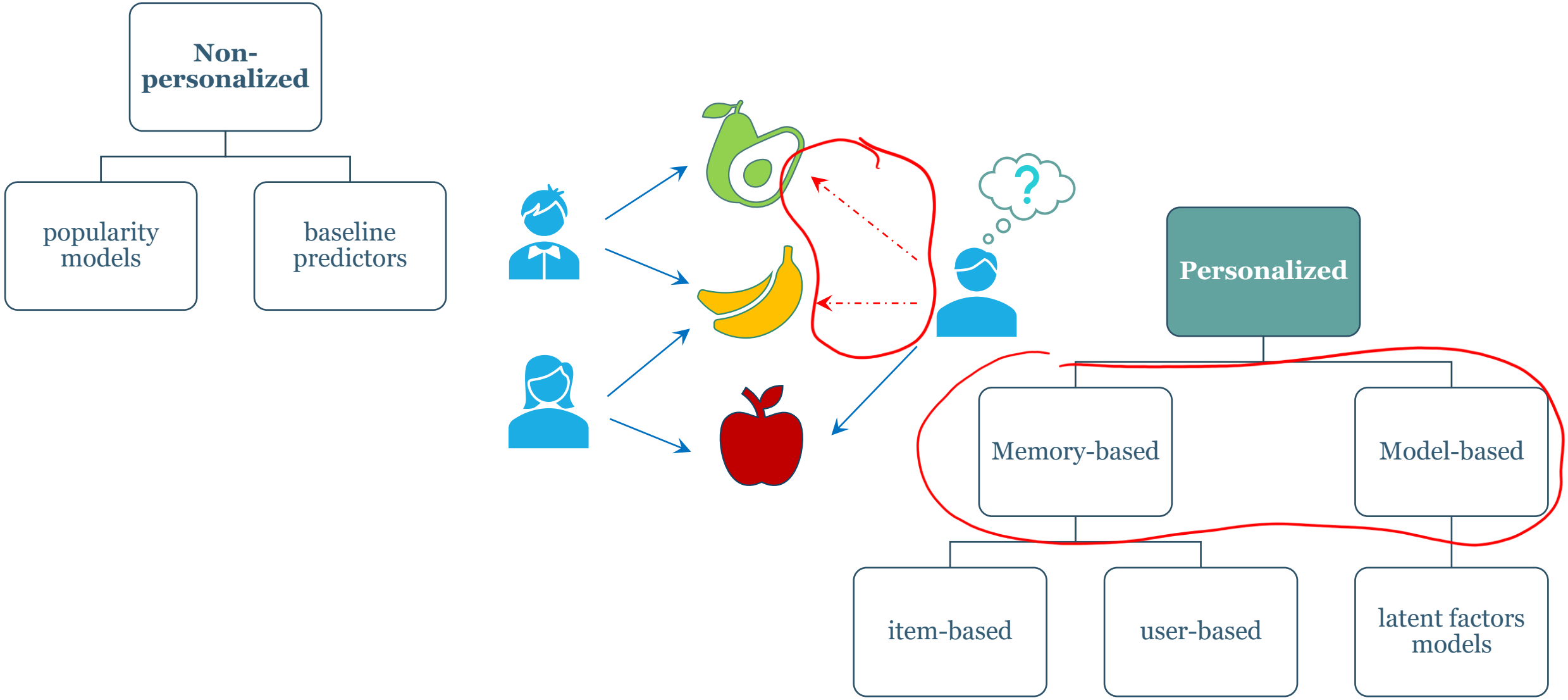
c: BeerAdvocate.

Image credit: "*On the robustness and discriminative power of information retrieval metrics for top-N recommendation*" Daniel Valcarce et al. In Proceedings of the 12th ACM Conference on Recommender Systems (2018) https://dl.acm.org/citation.cfm?doid=3240323.3240347

Collaborative Filtering

"wisdom of crowds"

Collaborative Filtering



General workflow

Goal: predict user preferences based on prior user feedback and collective user behavior.

collect data

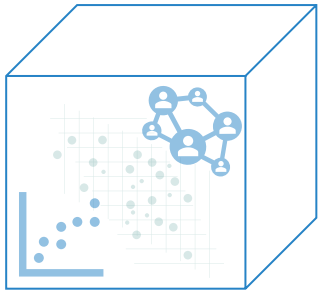
user-movie matrix A of size $M \times N$

a_{ij} is a rating of i^{th}user for j^{th} movie

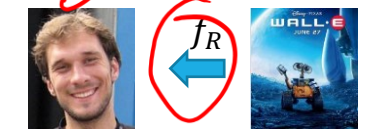
? - missing (unknown) values

build model

f_R: User × Item → Relevance



generate recommendations



unknown user

“Customers who like … also like …”

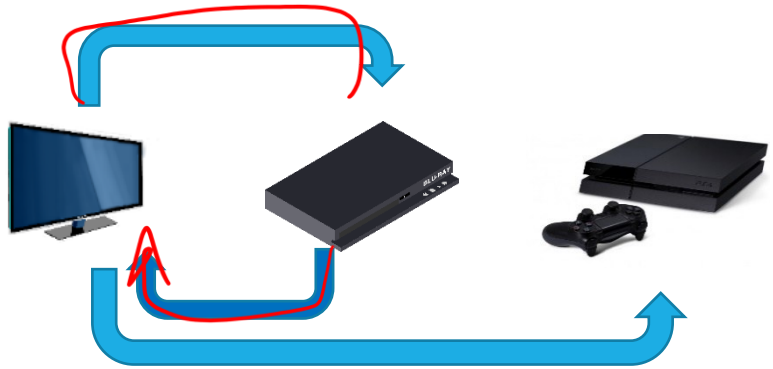
How do we implement that logic?

Pure item-to-item

Typical transactions log:

user id	item id	transact.
0	575	view
0	1881	view
0	846	basket
1	1878	purchase
1	576	view
…	…	…

Count co-occurrence of items:



$$\text{score}_{\text{I2I}}(u, i) = \sum_{\substack{j \in I_u \\ j \neq i}} \text{pairCount}(i, j)$$

I_u - set of items from user u history

Item-to-item is a strong baseline on very “sparse” datasets.

Computing I2I scores

For user-item interactions matrix A, the scores are:

$$C = A^\top A - \operatorname{diag}\left(\operatorname{diag}(A^\top A)\right)$$

If p is a vector of known user preferences,
then the vector of predicted relevance scores is:

$$r = Cp$$

Recommendations:

$$\operatorname{toprec}(n) := \arg\max_j^n r_j$$

Item-to-item issues

- somewhat obvious recommendations
  - high influence of popular items
- i2i matrix can also become dense if there are too many interactions per user

$N \times N$

CSR

Complexity analysis

$\equiv A^T A$ $\quad O(N^2 M)$ flops

$O(N^2)$ memory

$u \sim \log(N)$

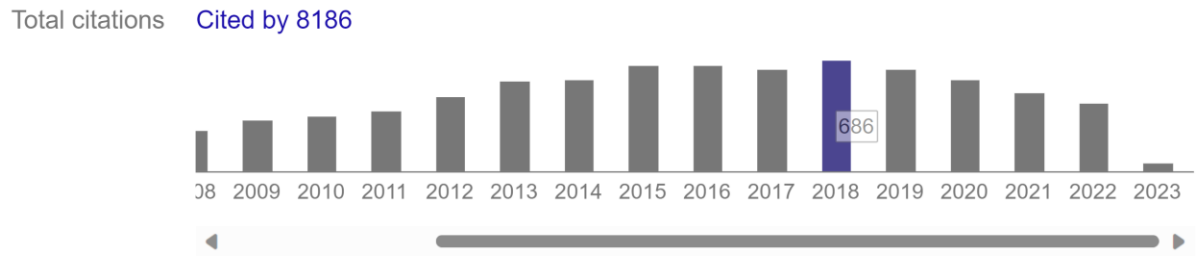
$O(N \log N)$ - mem.

$O(NM \log N)$ - flops

Case study: Amazon item-to-item

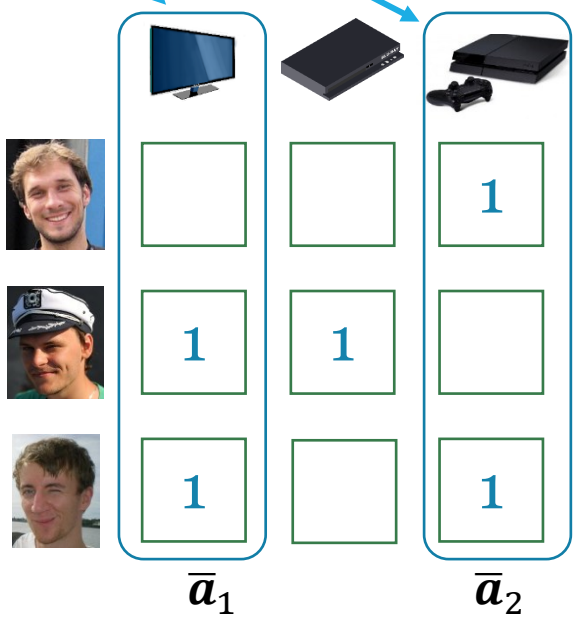
```
For each item in product catalog, I1
  For each customer C who purchased I1
    For each item I2 purchased by
      customer C
      Record that a customer purchased I1
      and I2
  For each item I2
    Compute the similarity between I1 and I2
```

G. Linden, B. Smith and J. York, "Amazon.com recommendations: item-to-item collaborative filtering," in IEEE Internet Computing, vol. 7, no. 1, pp. 76-80, Jan.-Feb. 2003.



Iterative algorithm

Computes similarity of items based on user purchases.



$$\text{sim}(I_1, I_2) = \cos(\bar{a}_1, \bar{a}_2) = \frac{(\bar{a}_1, \bar{a}_2)}{\|\bar{a}_1\|\|\bar{a}_2\|}$$

$\bar{a}_k$ - "one-hot" representation of item k

Recommender Systems

Lecture 2

Matrix Factorization

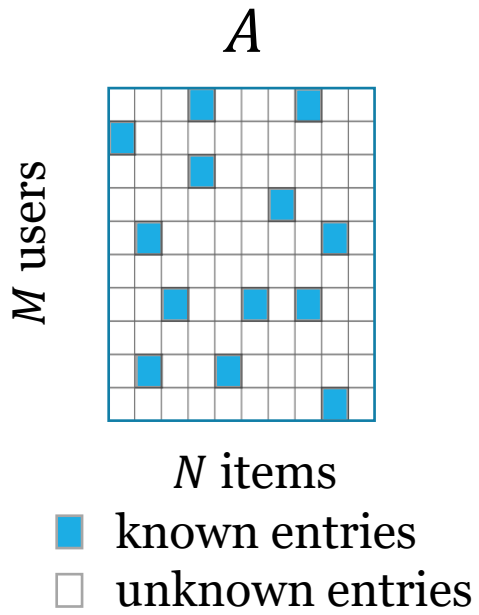
A general view on latent factors models

- **Task**: find utility (relevance) function f_R :

$$f_R: \text{Users} \times \text{Items} \rightarrow \text{Relevance score}$$

- As optimization problem with some *loss function* $\mathcal{L}$:

$$\mathcal{L}(A, R) \rightarrow \min$$



Components of the model:

- Utility function to generate R
- Optimization objective defined by $\mathcal{L}$
- Optimization method (algorithm)

What is the form of R and $\mathcal{L}$?

Low rank representation

?4	3	5	5
4	?3	5	5

$A = PQ^T$

$P = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ $Q = \begin{bmatrix} 4 \\ 3 \\ 5 \\ 5 \end{bmatrix}$

4	3	?		
?	3	5		
4	?	5		
			4	5
			?	5

Comedy Drama

$P = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}$ $Q = \begin{bmatrix} 4 & 0 \\ 3 & 0 \\ 5 & 0 \\ 0 & 4 \\ 0 & 5 \end{bmatrix}$

Intuition behind MF

Assumption: observed interactions can be explained via

- a *small* number of common patterns in human behavior
- + individual variations (including random factors and "unknown unknowns")

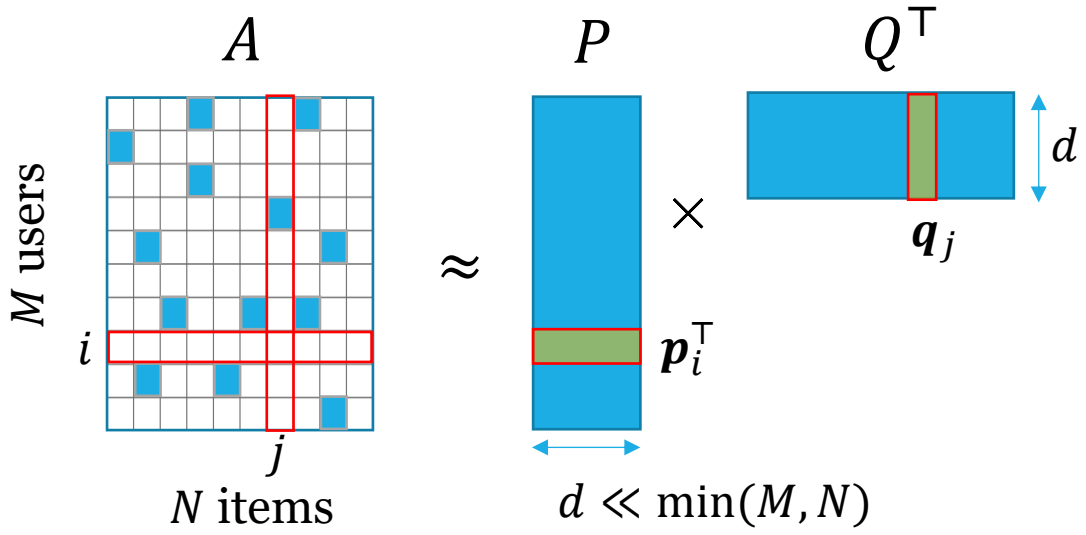
$$A_{full} = R + E, \qquad R = PQ^{\top}$$

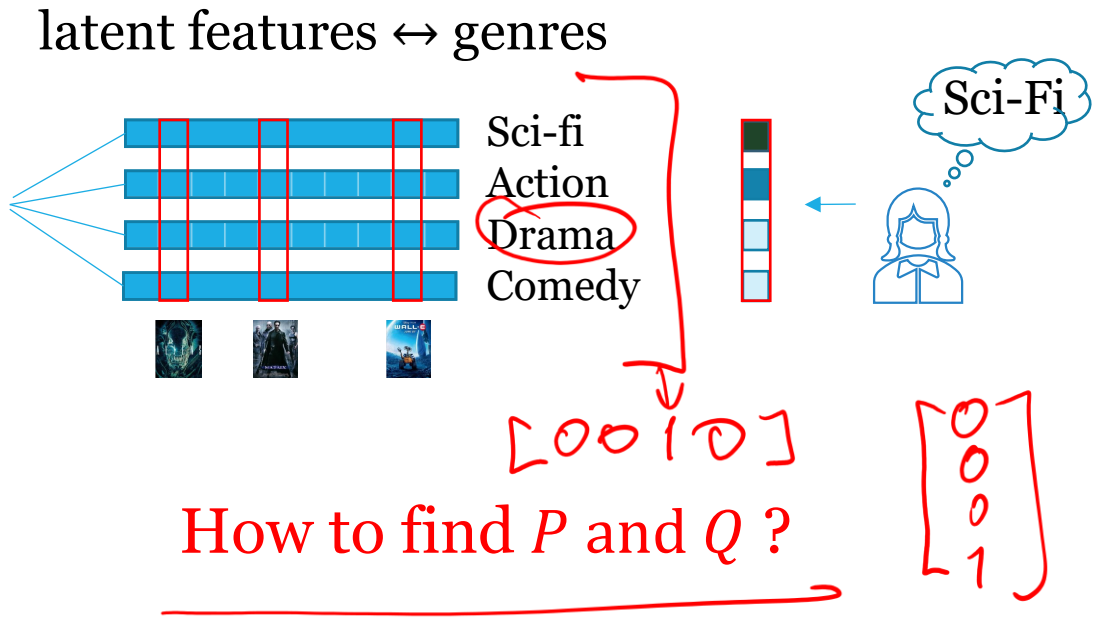
Predicted utility of item j for user i:

$$r_{ij} \approx \boldsymbol{p}_i^{\top} \boldsymbol{q}_j = \sum_{k=1}^{d} p_{ik} q_{jk}$$

$\boldsymbol{p}_i$ – latent factors vector for user i
$\boldsymbol{q}_j$ – latent factors vector for item j

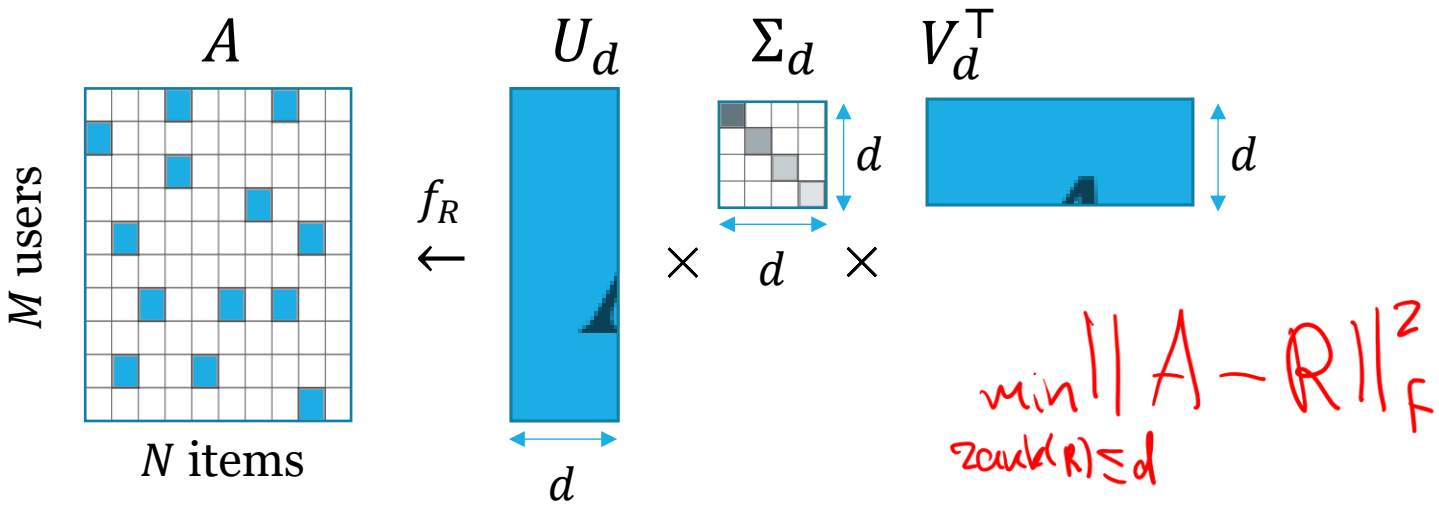
Simplistic view on latent features





rows of P – user embeddings
rows of Q – item embeddings

PureSVD model for CF



$$\min \|A - R\|_F^2$$
$$\text{rank}(R) \le d$$

$$A = U \Sigma V^T \quad \text{rank}(A) \le \min(M, N)$$
$$A \approx R = U_d \Sigma_d V_d^T$$
Σ - diag
$\sigma_1 \ge \sigma_2 \ge \dots \sigma_d \ge 0$

SVD is undefined for incomplete matrices.
Idea: let's impute zeros in place of unknowns!

$$A_0 = U \Sigma V^\top, \qquad [A_0]_{ij} = \begin{cases} a_{ij}, & \text{if known} \\ 0, & \text{otherwise} \end{cases}$$

$$R = U_d \Sigma_d V_d^\top$$

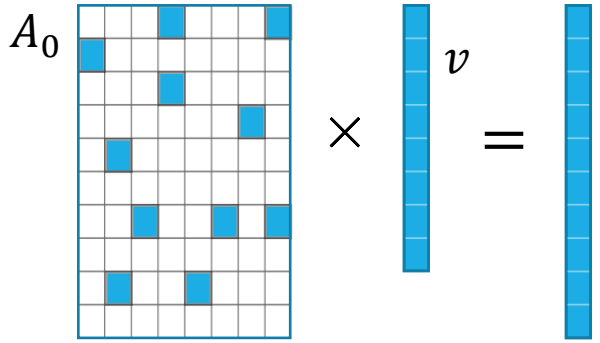
More convenient (equivalent) relevance score prediction:

$$R = A_0 V_d V_d^\top =$$
$$= U_d U_d^T A_0$$

PureSVD computation

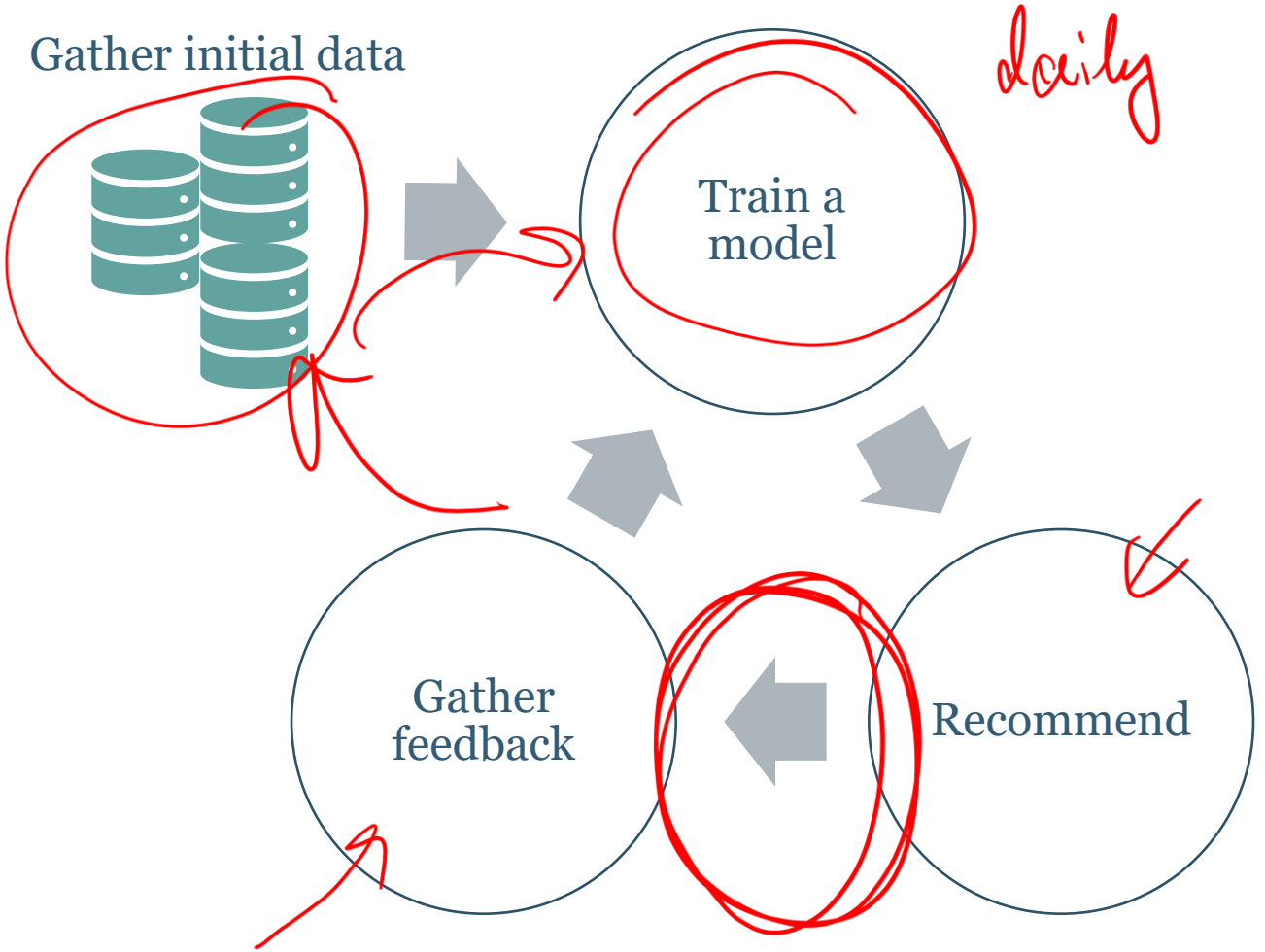
- Efficient computation with Lanczos algorithm
  - iterative process
  - requires only sparse matvec (fast with CSR format)
  - complexity $O(nnz \cdot d) + O((M+N) \cdot d^2)$

    nnz – number of non-zeros of A_0

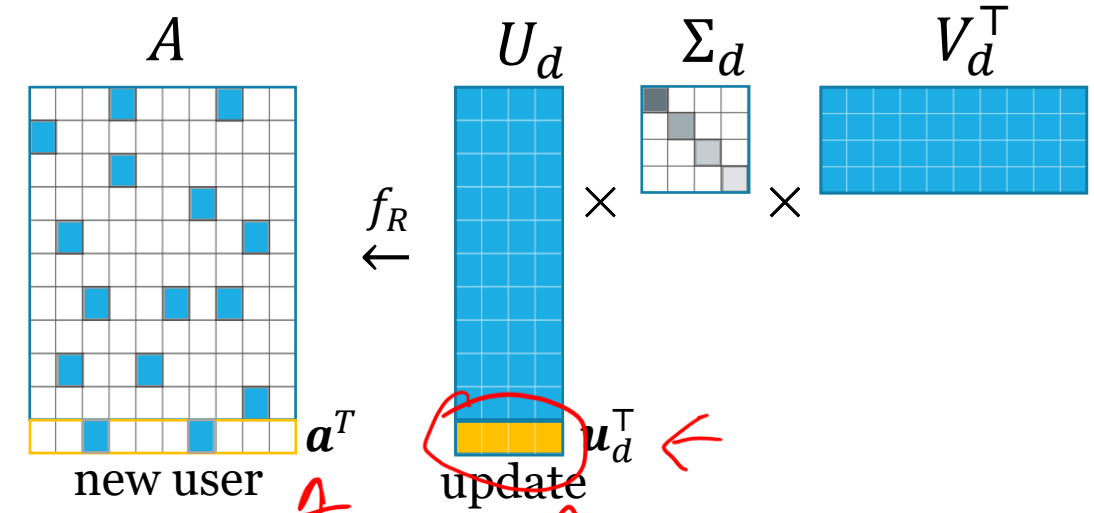


- Efficient implementations in Python:
  - SciPy Sparse svds,
  - Scikit-Learn TruncatedSVD.
- Core functionality is also implemented in Spark.

Lifecycle of a recsys model



PureSVD – recommending online



Finding a warm-start user representation:

$$\left\|\boldsymbol{a}_0^\top - \boldsymbol{u}^\top \Sigma V^\top\right\|_2^2 \to \min$$

new user embedding

$$\boldsymbol{u}^\top = \boldsymbol{a}_0^\top V \Sigma^{-1}$$

Prediction:

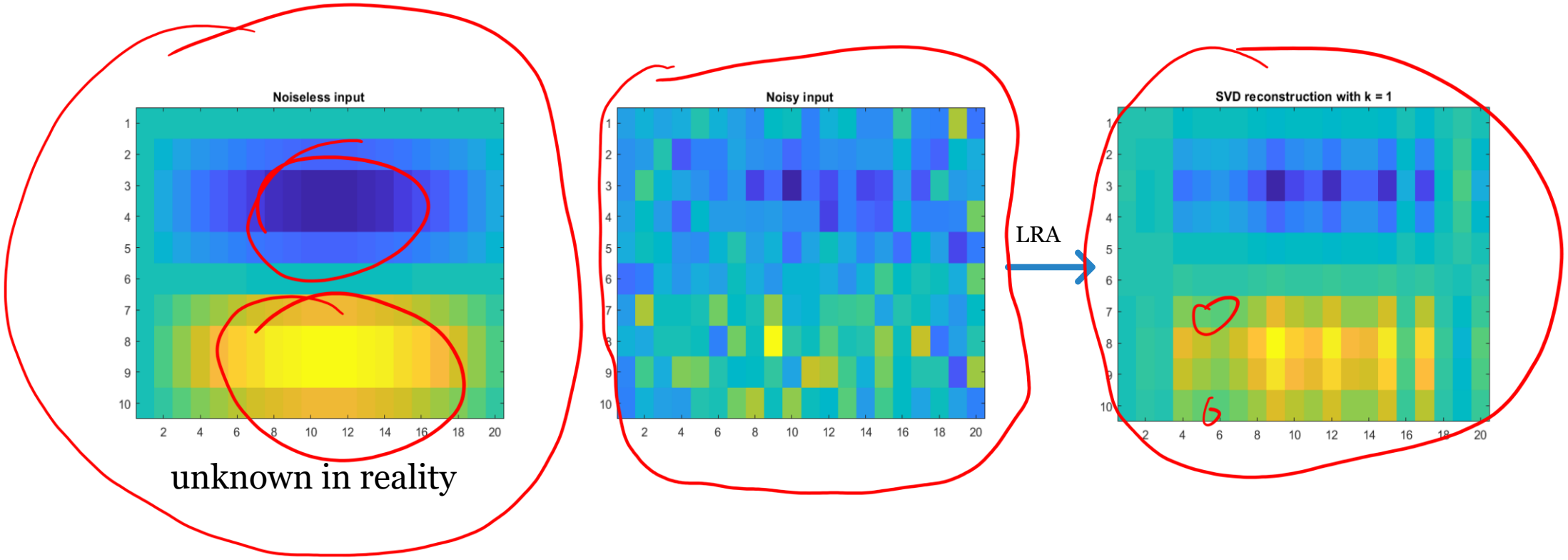
$$\boldsymbol{r}^\top = \boldsymbol{u}^\top \Sigma_d V_d^\top = \boldsymbol{a}_0^\top V_d V_d^\top$$

$$\boldsymbol{r} = V_d V_d^\top \boldsymbol{a}_0$$

- convenient for evaluation
- complexity $\sim O(Nd)$
- enables real-time recommendations

*G. Furnas, S. Deerwester, and S. Dumais, "Information Retrieval Using a Singular Value Decomposition Model of Latent Semantic Structure," Proceedings of ACM SIGIR Conference, 1988

CF as low-rank approximation task



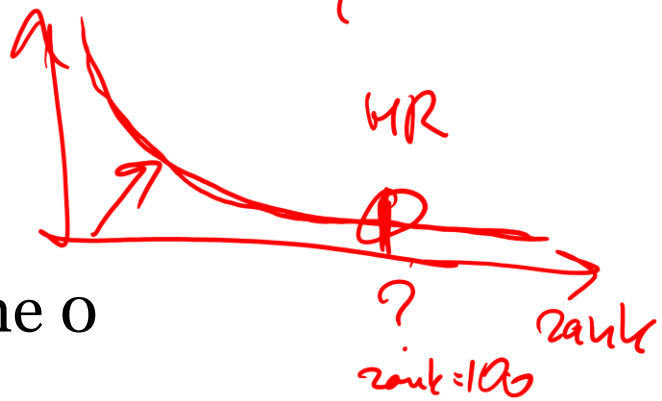
Images source: Khoshrou, Abdolrahman, and Eric J. Pauwels. "Regularisation for PCA-and SVD-type matrix factorisations." *arXiv preprint arXiv:2106.12955* (2021).

Approximation with irreducible noise

$$A = \boldsymbol{e} \cdot \boldsymbol{a}^{\top} + \epsilon B \qquad B_{ij} \sim \mathcal{N}(0,1)$$

$$\sigma_1^2(A) \leq M\|\boldsymbol{a}\|^2 + \epsilon^2 N, \qquad \sigma_k^2(A) \approx \epsilon^2 N, k \gg 1$$

?	3	5	5
4	?	5	5



Practical consequences:

- even in the simplest case singular values won't become 0
- no simple choice of the optimal rank of the decomposition

$$\left\|A - U_d \Sigma_d V_d^{\top}\right\|_{\mathrm{F}} = \sqrt{\sigma_{d+1}^2 + \cdots + \sigma_{\min(M,N)}^2}$$

- doesn't mean you can't get close to zero RMSE on trainset

Data centering (PCA style)

- in PureSVD, values are highly biased towards 0
- common observation from Netflix Prize era: a signal is carried mostly by baseline estimators

How does it affect rating prediction?

Strategy:

- value imputation → mean shifted matrix
- akin to data centering in PCA

Spectrum of mean-shifted ratings matrix

$$A = A_0 + \alpha \boldsymbol{e}_M \boldsymbol{e}_N^\top$$

$$\sigma_1^2(A) = \sigma_1^2(A_0) + \alpha^2 MN, \qquad \sigma_k^2(A) \approx \sigma_k^2(A_0), k \gg 1$$

Gives:

- More pronounced leading part of singular spectrum
- Better RMSE
- Lower accuracy

Practical consequences

- SVD for CF is:
  - NOT pure matrix completion
  - NOT pure dimensionality reduction
- common PCA-like preprocessing may spoil data representation
- rating prediction doesn't make sense
  - recommendations can still be good!
  - we can treat rating values more flexibly

Mitigating popularity bias

Data normalization in PureSVD

- Interactions data approximately follows power-law or zipf-like distributions.
- What effect does it have on covariance matrix?
  - $\bar{\boldsymbol{a}}_i^\top \bar{\boldsymbol{a}}_j$
- Idea: normalization inversely proportional to popularity

$$\tilde{A} = AD^{f-1}, \qquad [D]_{ii} = \|\bar{\boldsymbol{a}}_i\|$$

- Other options: user-based normalization or combined`

Weighted Matrix Factorization

Weighted Matrix Factorization

Previously:

- How to handle incomplete data?
- PureSVD: just put 0

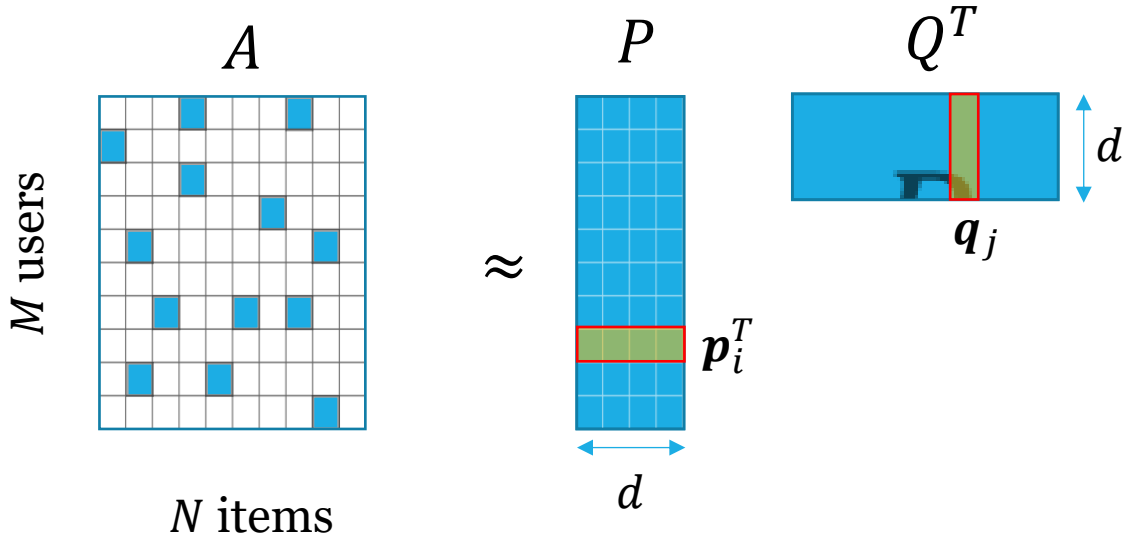
Is there a better way?

Weighted Matrix Factorization

Loss function:

$$\mathcal{L}(\mathrm{A}, \Theta) = \frac{1}{2} \sum_{i,j \in \mathcal{O}} \left(a_{ij} - \boldsymbol{p}_i^\top \boldsymbol{q}_j\right)^2$$

$$\mathcal{O} = \{(i,j):\ a_{ij} \text{ is known}\}$$



Matrix form:

$$\mathcal{L}(\mathrm{A}, \Theta) = \|W \odot (A - R)\|_F^2$$

$$R = PQ^\top$$

$\odot$ - Hadamard product

simplest case - binary weights:

$$\begin{cases} w_{ij} = 1, & \text{if } a_{ij} \text{ is known,} \\ w_{ij} = 0, & \text{otherwise.} \end{cases}$$

Recall PureSVD from: $\mathcal{L} = \|A_0 - R\|_F^2$

MF optimization objective

Optimization objective:

loss

Regularization

$$\mathcal{J}(\Theta) = \mathcal{L}(\mathrm{A}, \Theta) + \Omega(\Theta)$$

Model parameters: $\Theta = \{P, Q\}$

$\Omega(\Theta)$ - additional constraints, e.g. L_2 regularization

Typical optimization algorithms:

- stochastic gradient descent (SGD)
- alternating least squares (ALS)

ALS:

$$\begin{cases} P^* = \arg\min_P \mathcal{J}(\Theta) \\ Q^* = \arg\min_Q \mathcal{J}(\Theta) \end{cases}$$

GD:

$$\begin{cases} \boldsymbol{p}_i \leftarrow \boldsymbol{p}_i - \eta \nabla_{\boldsymbol{p}_i} \mathcal{J} \\ \boldsymbol{q}_j \leftarrow \boldsymbol{q}_j - \eta \nabla_{\boldsymbol{q}_j} \mathcal{J} \end{cases}$$

Optimization with Alternating Least Squares

ALS:

$$\begin{cases} P = \arg\min_P \mathcal{J}(\Theta) \\ Q = \arg\min_Q \mathcal{J}(\Theta) \end{cases}$$

$$\mathcal{J}(\Theta) = \mathcal{L}(\Theta) + \Omega(\Theta)$$

$$\mathcal{L}(\Theta) = \frac{1}{2}\left\| W \odot (A - PQ^\top) \right\|_F^2$$

$$\Omega(\Theta) = \frac{1}{2}\lambda\left(\|P\|_F^2 + \|Q\|_F^2 \right)$$

The optimization problem is bi-convex:

"user-oriented" form:

$$\mathcal{J}(\Theta) = \frac{1}{2}\sum_i \|\boldsymbol{a}_i - Q\boldsymbol{p}_i\|_{W^{(i)}}^2 + \frac{1}{2}\lambda \sum_i \|\boldsymbol{p}_i\|_2^2 + \frac{1}{2}\lambda \|Q\|_F^2$$

notation: $\|\boldsymbol{x}\|_W^2 = \boldsymbol{x}^\top W \boldsymbol{x}$

$W^{(i)} = \text{diag}\{w_{i1}, w_{i2}, \ldots, w_{iN}\}$

Optimization with ALS

$$\mathcal{J}(\Theta) = \frac{1}{2}\sum_i \|\boldsymbol{a}_i - Q\boldsymbol{p}_i\|^2_{W^{(i)}} + \frac{1}{2}\lambda \sum_i \|\boldsymbol{p}_i\|_2^2 + \frac{1}{2}\lambda \|Q\|_{\mathrm{F}}^2$$

$$\|\boldsymbol{x}\|_W^2 = \boldsymbol{x}^\top W \boldsymbol{x}$$

$$W^{(i)} = \mathrm{diag}\{w_{i1}, w_{i2}, \ldots, w_{iN}\}$$

$$\frac{\partial \mathcal{J}(\Theta)}{\partial P} = 0$$

$\frac{\partial}{\partial p_i}$:

$$(Q^\top W^{(i)} Q + \lambda I)\,\boldsymbol{p}_i = Q^\top W^{(i)} \boldsymbol{a}_i$$

$B_x = 8$

$B \sim d \times d$ $\quad O(d^3)$

Optimization with ALS

$$\mathcal{J}(\Theta) = \frac{1}{2}\sum_i \|\boldsymbol{a}_i - Q\boldsymbol{p}_i\|^2_{W^{(i)}} + \frac{1}{2}\lambda \sum_i \|\boldsymbol{p}_i\|_2^2 + \frac{1}{2}\lambda \|Q\|_F^2$$

$$\|\boldsymbol{x}\|^2_W = \boldsymbol{x}^\top W \boldsymbol{x}$$

$$W^{(i)} = \text{diag}\{w_{i1}, w_{i2}, \ldots, w_{iN}\}$$
$$\bar{W}^{(j)} = \text{diag}\{w_{1j}, w_{2j}, \ldots, w_{Mj}\}$$

$$\frac{\partial \mathcal{J}(\Theta)}{\partial P} = 0$$
$$\frac{\partial \mathcal{J}(\Theta)}{\partial Q} = 0$$

entity-wise updates

Block-coordinate descent:

$$(Q^\top W^{(i)} Q + \lambda I)\, \boldsymbol{p}_i = Q^\top W^{(i)} \boldsymbol{a}_i$$

$$(P^\top \bar{W}^{(j)} P + \lambda I)\, \boldsymbol{q}_j = P^\top \bar{W}^{(j)} \bar{\boldsymbol{a}}_j$$

$d \times N \quad N \times N \quad N \sim nnz_A$

system of linear equations: $A\boldsymbol{x} = \boldsymbol{b}$

$O((N+M)\, d^3)$

"Embarrassingly parallel" algorithm

Initialize P and Q.

Iterate until stopping criteria met:

$P \leftarrow \arg\min_P \mathcal{J}(\Theta)$

$Q \leftarrow \arg\min_Q \mathcal{J}(\Theta)$

https://en.wikipedia.org/wiki/Embarrassingly_parallel

ALS performance

Complexity:

$$O(\qquad\qquad) + O(\qquad\qquad)$$

$z = Q_p$

$$(Q^\top W^{(i)} Q + \lambda I)\, \boldsymbol{p}_i = Q^\top W^{(i)} \boldsymbol{a}_i$$

ALS performance

Complexity:

$$O(nnz_A \cdot d^2) + O((M + N) \cdot d^3)$$

- can be improved with approximate solvers (e.g. conjugate gradient)
- also fast implementations using coordinate descent method

ALS folding-in

trivial

Folding-in and timepoint splits

consider scenario:

- warm-start users for evaluation
- test users are the most recently active ones
- most-recent-item holdout

Will there be a data leakage?

Case study: Yandex Zen (old times)

Company manages many different types of media content (news, search, etc.).

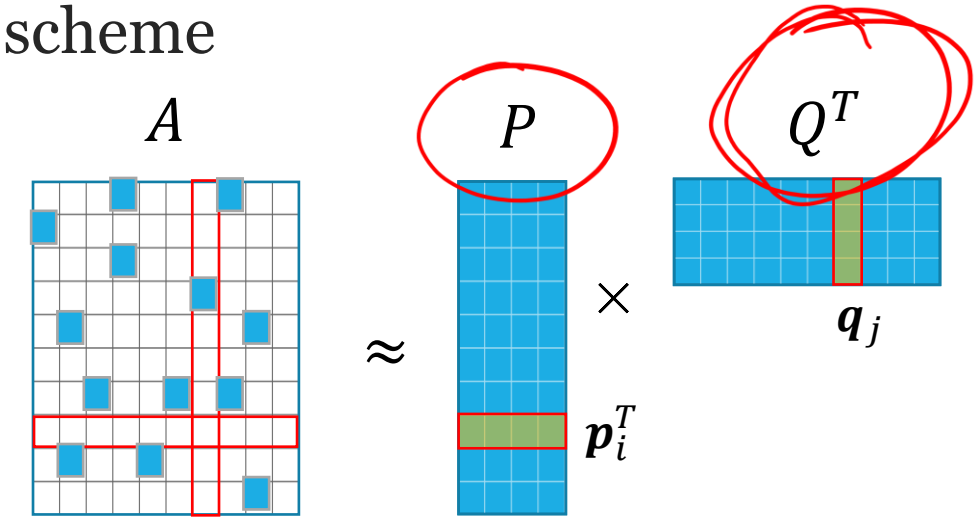
Goal: have a unified user representation across all domains.

Solution:

- A neural network embeds onto a latent space all unstructured content
- Users are updated through the “half”-ALS scheme

Algorithm:

- Get Q from external source (ANN)
- Update P based on most recent Q

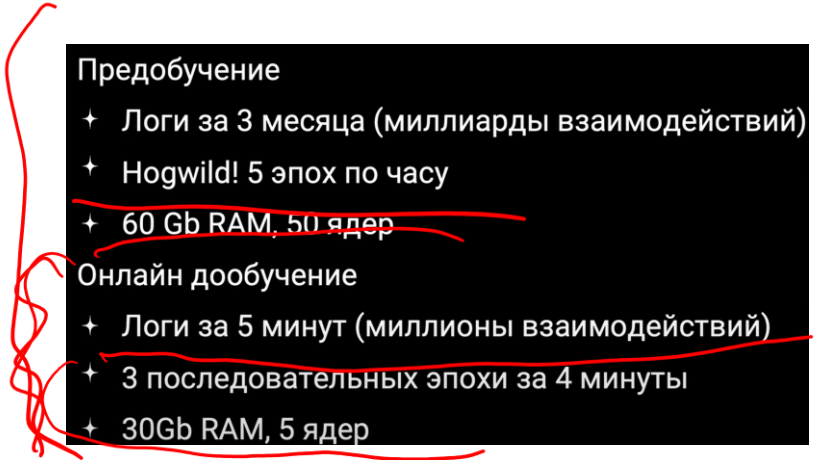


Case study: Yandex Zen (recent times)

- fully MF-based solution, no neural networks
- solution beats more complex approaches
- SGD-based approach with additional tricks
  - pre-training on larger datasets (prior history)
  - downweight embeddings of “cold” users and items at initialization
  - fp16 calculations

Talk (in Russian)

- Ренессанс факторизации в рекомендациях
  https://www.youtube.com/watch?v=3Inl0iE41NU&list=PLfaEB9oj-8KVBZNFcrESLBGOc5joIxCzK&index=2

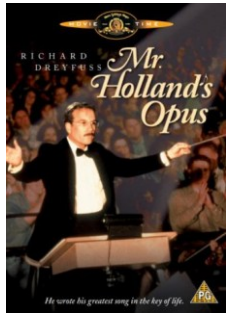


Many facets of user feedback

Guess standard algorithm behavior

What is likely to be recommended in this case?

User feedback is negative!
Probably he or she doesn't like criminal movies.



Problematic warm start scenario

f_R

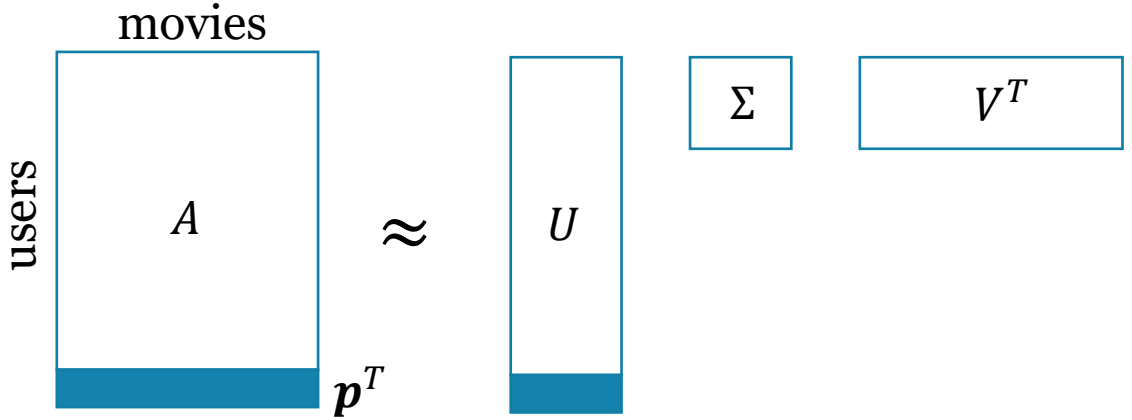
recommendations are insensitive to negative feedback

Traditional models unable to recommend relevant items for new user if given low rated examples only.

Explanation

			
Alice	2	5	3
Bob	4		5
Carol	2	5	
Tom	2	???	???

MF prediction



predicted scores (*folding-in*):

$$r \approx VV^T p$$

top-*n* recommendations task:

$$\text{toprec}(p, n) := \arg\max_n r$$

$$\arg\max VV^T(0, \ldots, 0, \mathbf{2}, 0, \ldots, 0)^T \equiv \arg\max VV^T(0, \ldots, 0, \mathbf{5}, 0, \ldots, 0)^T$$

★★☆☆☆ ★★★★★

Rating value doesn't change ranking of the items!

Case study: (old) Kinopoisk

Some user's "know-how" on improving the quality of recommendations.

Я склонен согласиться с этими утверждениями, потому что у меня оценки перевалили за 2000, и на протяжении их выставления я замечаю как **деградируют** рекомендации. Интуитивно (а так же с помощью прикладного ПО) я ощущаю, что **высокие оценки играют бо'льшую роль в рекомендации фильма**, так же обеими руками за 2-е утверждение, **спектр моих положительных эмоций гораздо шире, чем отрицательных, я часто сомневаюсь между 7 и 8, но почти никогда не заморачивался над 2 или 3**. Однако, я не согласен, что низкие оценки не надо учитывать вообще, я решил проблему тем, что удалил большую половину низких оценок. Задумайтесь, ведь большинство низких оценок - это недосмотренные или промотанные фильмы, часто вы даже сюжета не вспомните из них, только пару сцен, это не повод ставить оценку. **Если вы хотите качественных рекомендаций, не гонитесь за количеством оценок** (сам когда-то был грешен, понаоценивал мексиканских сериалов, о которых слышал, когда пешком под стол ходил).

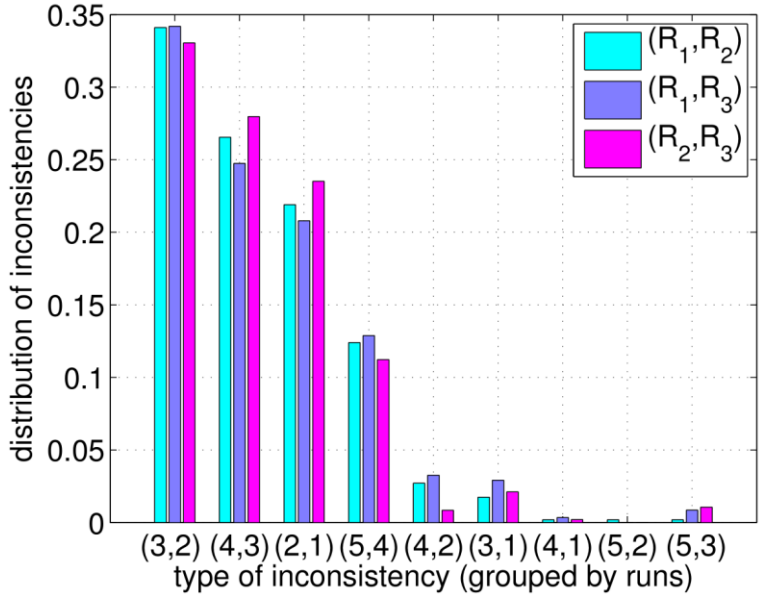
https://forum.kinopoisk.ru/showthread.php?t=57341 (no longer exists)

User feedback peculiarities

From neoclassical economics:
utility is an **ordinal concept**.

Traditional recommender models treat ratings as **cardinal numbers**.

Ratings scale consist of **unequal intervals***:



*"I like it... I like it not: Evaluating User Ratings Noise in Recommender Systems" by Xavier Amatriain, Josep M. Pujol, and Nuria Oliver

Combining implicit and explicit feedback

Previously:

$$\mathcal{J}(P,Q) = \frac{1}{2}\sum_{i,j=1}^{M,N} w_{ij}\left(a_{ij} - \boldsymbol{p}_i^\top \boldsymbol{q}_j\right)^2 + \lambda\left(\|\boldsymbol{p}_i\|^2 + \|\boldsymbol{q}_j\|^2\right)$$

$$w_{ij} = \begin{cases} 1, & \text{if } a_{ij} \text{ is known,} \\ 0, & \text{otherwise.} \end{cases}$$

Recall,

- ratings are not what we want to predict, but
- ratings still indicate utility/relevance of an item;
- we generally want better prediction accuracy on higher ratings

What if $w_{ij} = f(a_{ij})$ would be a non-trivial function of feedback?

Combining implicit and explicit feedback

New formulation:

$$\mathcal{J}(P,Q) = \frac{1}{2}\sum_{i,j=1}^{M,N} w_{ij}\left(s_{ij} - \boldsymbol{p}_i^\top \boldsymbol{q}_j\right)^2 + \lambda\left(\|\boldsymbol{p}_i\|^2 + \|\boldsymbol{q}_j\|^2\right)$$

$$w_{ij} = f(a_{ij}), \qquad s_{ij} = \begin{cases} 1, & \text{if } a_{ij} \text{ is known,} \\ 0, & \text{otherwise.} \end{cases}$$

$w_{ij} = \text{const}$ a_{ij} unknown

Predicting implicit feedback with explicit-feedback-based weighting

Confidence-based model (a.k.a iALS / WRMF)

$$\mathcal{J}(\Theta) = \mathcal{L}(\Theta) + \Omega(\Theta)$$

$$\mathcal{L}(\Theta) = \frac{1}{2}\left\|W \odot (S - PQ^{\top})\right\|_{\mathrm{F}}^{2}, \qquad S: \begin{cases} s_{ij} = 1, & \text{if } a_{ij} \text{ is known,} \\ s_{ij} = 0, & \text{otherwise.} \end{cases}$$

Weights $W = \left[w_{ij}^{1/2}\right]$ are not binary anymore:

$$w_{ij} = \begin{cases} 1 + \alpha f(a_{ij}), & \text{if } a_{ij} \text{ is known,} \\ 1, & \text{otherwise.} \end{cases}$$

Examples of weighting function: $f(x) = x$ or $f(x) = \log\left(x + \frac{1}{\epsilon}\right)$

Complexity of iALS updates

$$\mathcal{J}(\Theta) = \frac{1}{2}\sum_i \|\boldsymbol{s}_i - Q\boldsymbol{p}_i\|^2_{W^{(i)}} + \frac{1}{2}\lambda \sum_i \|\boldsymbol{p}_i\|_2^2 + \frac{1}{2}\lambda \|Q\|_{\mathrm{F}}^2$$

$W^{(i)}$ is diagonal matrix of size $N \times N$ with elements $w_{i1} \ldots w_{iN}$

Complexity of iALS updates

$$\mathcal{J}(\Theta) = \frac{1}{2}\sum_{i=1}^{M} \|\boldsymbol{s}_i - Q\boldsymbol{p}_i\|^2_{W^{(i)}} + \frac{1}{2}\lambda \sum_{i=1}^{M} \|\boldsymbol{p}_i\|_2^2 + \frac{1}{2}\lambda \|Q\|_{\mathrm{F}}^2$$

$W^{(i)} = \mathrm{diag}\{w_{i1}, \ldots, w_{iN}\}$ is $N \times N$ diagonal matrix

$$\boldsymbol{p}_i = (Q^\top W^{(i)} Q + \lambda I)^{-1} Q^\top W^{(i)} \boldsymbol{s}_i$$

Complexity of updates in naïve approach:

$$\mathrm{O}(MN \cdot d^2) + \mathrm{O}((M+N)d^3)$$

iALS performance

$$\boldsymbol{p}_i = (Q^\top W^{(i)} Q + \lambda I)^{-1} Q^\top W^{(i)} \boldsymbol{s}_i$$

$$w_{ij} = \begin{cases} 1 + c_{ij}(a_{ij}), & \text{if } a_{ij} \text{ is known,} \\ 1, & \text{otherwise.} \end{cases}$$

const

$$W^{(i)} = I + C^{(i)}$$

nnz

Trick: pre-calculate and store $d \times d$ matrix $Q^\top Q$

$$Q^\top W^{(i)} Q = Q^\top Q + Q^\top C^{(i)} Q$$

Overall complexity reduces to:

$$O\left(nnz_A \cdot d^2\right) + O\left((M + N) \cdot d^3\right)$$

Case study: Yandex Music

https://academy.yandex.ru/posts/kholodnye-polzovateli-i-mnogorukie-bandity

Need to take into account different signals! For example:

- number of seconds a user spent listening to a track
- number of skips / fast forwards
- adding to playlist / favorites
- …

Solved by iALS / WRMF model

$$\mathcal{L} = \sum_{ij} w(a_{ij}) \cdot l(s_{ij} - r_{ij})$$

Confidence for negative samples

- Standard confidence-based scenario:
  - higher weights on errors *over seen interactions*, constant on negatives
- "Inverted" case:
  - lower weights on errors *over unseen interactions*, constant on positives
  - the confidence of missing values being negative is not as high as of non-missing values being positive
  - possible cases:
    - missing feedback of an active user is more likely to be true negative, $w_{ui} \propto |\mathcal{I}_u|$
    - missing feedback on a niche item is more likely to be true negative, $w_{ui} \propto M - |\mathcal{U}_i|$
  - Alternative view:
    - miss on a popular item is more likely to be true negative, $w_{ui} \propto f(|\mathcal{U}_i|)$

Weighting strategies recap

	$(u,i) \in \mathcal{O}$	$(u,i) \in (\mathcal{U} \times \mathcal{I}) \setminus \mathcal{O}$	reference
iALS	$1 + \alpha f(r_{ui})$	1	Hu et al. [2008]
uniform	1	$0 \leq \text{const} < 1$	Pan et al. [2008]
user-oriented	1	$\alpha \lvert\mathcal{I}_u\rvert$	Pan et al. [2008]
item-oriented	1	$\alpha(M - \lvert\mathcal{U}_i\rvert)$	Pan et al. [2008]
item popularity	1	$c_0 \frac{f_i^\alpha}{\sum_{j=1}^N f_j^\alpha}, f_i = \frac{\lvert\mathcal{U}_i\rvert}{\sum_{j=1}^N \lvert\mathcal{U}_j\rvert}$	He et al. [2016] Sayapin et al. [2023]

Table inspired by https://ddoeunn.github.io/2021/05/02/SUMMARY-Weighted-Matrix-Factorization-for-Implicit-Feedback_2.html

Note on negative sampling

- SGD:
  - negative sampling
- iALS:
  - confidence weights
- PureSVD
  - data normalization

All these methods aim to balance contribution of positive and negative items!

Scalability of MF algorithms

- PureSVD:
  - Randomized SVD for large-scale data
    - Criteo
    - Facebook's RSVD
  - streaming and 0-communication versions of SVD
  - "exact" folding in ([Brand 2002])
- ALS:
  - embarrassingly parallel
  - but communication cost can be too high
  - Coordinate Descent can be a good alternative (e.g., eALS by [He et al. 2016])
- SGD:
  - inherently sequential
    - Hogwild! algorithm is not directly applicable in CF settings
  - there're tricks to run in parallel within sub-blocks of rating matrix
  - slow convergence in general
    - using adaptive learning rate schedule (ADAM, Adagard, etc.) helps

Algorithm	Overall complexity	Update complexity	Sensitivity
SVD*	$O\left(nnz_A \cdot r + (M+N)r^2\right)$	$O(nnz_a \cdot r)$	Stable
ALS	$O\left(nnz_A \cdot r^2 + (M+N)r^3\right)$	$O\left(nnz_a \cdot r + r^3\right)$	Stable
CD	$O(nnz_A \cdot r)$	$O(nnz_a \cdot r)$	Stable
SGD	$O(nnz_A \cdot r)$	$O(nnz_a \cdot r)$	Sensitive

* For both standard and randomized implementations [71].

General rule of thumb: avoid distributed setups.

ALS vs SGD vs SVD

ALS

- More stable
- Fewer hyper-parameters to tune
- Higher complexity, however requires fewer iterations
- Embarrassingly parallel
- Higher communication cost in distributed environment

SGD

- Sensitive to hyper-parameters
- Requires special treatment of learning rate
- Lower complexity, but slower convergence
- Inherently sequential (parallelization is tricky for RecSys)

Unlike SVD:

More involved optimization (no rank truncation).
No global convergence.
Allow for custom optimization objectives. $\mathcal{L}(A, R) \rightarrow \mathcal{L}(f(A, R))$